DevEx with Red Hat Developer Hub

Version 1, 2026-10-26

DevEx with Developer Hub

Introduction

Welcome to this quick course on the ***DevEx with Red Hat Developer Hub***. This course is the `fourth` in a series of `six` courses about **Application Platform Delivery**:

- *Introduction to Application Platforms and Platform Engineering*
- *Software Factory Foundations with OpenShift Dev Spaces*
- *Software Delivery with OpenShift Pipelines and GitOps*
- *DevEx with Red Hat Developer Hub* (*This course*)
- *Application Modernization with MTA*
- *Trusted Software Supply Chain (TSSC)* (Coming Soon)

What's in this course?

In this course, you'll learn how **Red Hat Developer Hub** provides a unified **Internal Developer Portal(IDP)** for a Software Factory. You'll explore Developer Experience, Software Templates (Golden Paths), the Software Catalog, platform integrations, and OpenShift OAuth authentication to enable secure, standardized, and self-service application delivery.

Duration: 2 hours

Contributors

The PTL team acknowledges the valuable contributions of the following Red Hat associates:

- Andrew Block (SME)
- Bill Wheatley (SME)
- Bryan Saunders (SME)
- Evan Shortiss (SME)
- Julio Villarreal Pelegrino (SME)
- Kristopher Godek (SME)
- Roman Martin Gil (SME)
- Senthil Kumar (SME)
- Sergio Canales Espinoza (SME)
- Yogita Soni (Content Architect)
- Lisa Edgell (Learning Product Manager)
- Caitlyn Rymarchyk (Learning Product Manager)
- Anna Swicegood (Learning Product Manager)

Learning Objectives

By the end of this module, you will be able to:

- Deploy and configure Red Hat Developer Hub on OpenShift.
- Create and register Software Templates (Golden Paths).
- Enable software discovery using the Software Catalog.
- Integrate templates with platform services and development tools.
- Configure OpenShift OAuth for secure authentication.
- Enable self-service application creation through Developer Hub.

Prerequisites

This course assumes that you have the following prior experience:

1. Red Hat OpenShift Technical Overview (DO080) v4.20 or Kubernetes Basics

Lab Environments

No new lab environment is required for this module. Continue using the **same RHDP lab environment** that you provisioned in ***Software Factory Foundations with OpenShift Dev Spaces***.

The environment includes the **"Service Enablement: App Platform Software Factory** and **"Service Enablement: App Platform Runtime"** labs required for this module.

Do **not** provision, delete, reset, or reprovision the environment. The labs in this module build on the applications, configurations, and platform resources created in ***Software Factory Foundations with OpenShift Dev Spaces***.

The cluster URLs and credentials are available from the **My Services** page in RHDP.

OpenShift Clusters

Environment	Purpose	Used In
Service Enablement: App Platform Software Factory (Primary Cluster)	Development platform where you'll build the internal developer platform, configure CI/CD, and deploy applications.	• *Software Factory Foundations with OpenShift Dev Spaces* • *Software Delivery with OpenShift Pipelines and GitOps* • *DevEx with Red Hat Developer Hub* • *Application Modernization with MTA*
Service Enablement: App Platform Runtime (Secondary Cluster)	Runtime environment used to demonstrate GitOps-based promotion between environments.	• *Software Delivery with OpenShift Pipelines and GitOps*

The required namespaces have already been created and are ready for use.

Internal Lab Account Credentials

Use the following accounts throughout the learning path.

Username	Password	Purpose
`platform-admin`	`openshift`	Platform administrator
`platform-developer`	`openshift`	Application developer
`platform-user`	`openshift`	Standard user

Unless instructed otherwise, authenticate to GitLab, Quay, Vault, and other integrated services using

Keycloak Single Sign-On (SSO).

Platform Services

The following services are pre-configured and available throughout the labs.

Service	Purpose
Red Hat Build of Keycloak	Provides centralized authentication and Single Sign-On (SSO).
GitLab	Hosts the application source code and Git repositories.
Red Hat Quay	Stores container images built during the labs.
OpenShift GitOps	Continuously synchronizes applications and platform resources from Git.
HashiCorp Vault	Securely stores secrets used by applications and CI/CD pipelines.

All service URLs are available from the **Red Hat Demo Platform (RHDP)** **Lab Information** in **My Services** section page.

Red Hat Developer Hub and the IDP



What's in this section?

In this section, you'll learn how to deploy and configure **Red Hat Developer Hub** as the front door to your Software Factory.

You'll install Developer Hub on OpenShift, create and register your first **Software Template (Golden Path)**, enable the **Software Catalog** for software discovery, and integrate enterprise authentication using **OpenShift OAuth**.

By the end of this section, you'll have a working Internal Developer Portal that allows developers to securely discover software, create new applications through self-service templates, and access the Software Factory through a single, unified experience.

Developer Experience Matters

Why Developer Experience Matters

- Modern software development requires developers to work with numerous tools throughout the Software Development Lifecycle (SDLC).
- A typical developer interacts with:
  - Source code repositories
  - Development environments
  - CI/CD pipelines
  - GitOps deployment tools
  - Container registries
  - Documentation portals
  - Security scanners
  - Monitoring and observability platforms



- Each tool solves a specific problem. However, switching between multiple tools increases the amount of information developers must understand before they can deliver software.
- This additional mental effort is commonly referred to as **developer cognitive load**.
- Platform Engineering aims to reduce this cognitive load by providing standardized workflows and self-service capabilities, allowing developers to spend more time writing applications and less time managing platform complexity.



Great platforms are not measured by the number of tools they provide, but by how simple they make software delivery for developers.

What is Developer Experience?



- *Developer Experience (DevEx)* describes the overall experience developers have while building, testing, deploying, and operating software.
- It encompasses the tools, workflows, platforms, environments, and organizational practices that influence how efficiently developers can deliver applications.

> Developer Experience is the user experience developers have while

> interacting with a platform, its tools, workflows, and supporting services.

- The primary goals of DevEx are to:
  - Reduce friction during software development.
  - Improve developer productivity.
  - Standardize development workflows.
  - Enable developers to focus on delivering business value.

Benefits of Improving Developer Experience

Improving Developer Experience benefits both developers and organizations.

Technical Benefits

- Reduced cognitive load
- Standardized development workflows
- Faster onboarding of new developers
- Faster software delivery
- Improved software quality
- Consistent engineering practices

Business Benefits

- Higher developer productivity
- Faster time to market
- Lower software development costs
- Reduced developer burnout
- Higher employee retention
- Increased innovation



Organizations that invest in Developer Experience often observe improvements in software delivery performance, developer satisfaction, and overall business outcomes.

Developer Experience and Platform Engineering

Developer Experience is closely related to several Platform Engineering concepts.

Platform Engineering

Builds, operates, and maintains the internal platform that enables developers to build and deliver software efficiently.

Internal Developer Platform (IDP)

A collection of standardized tools, automation, services, and workflows that developers use throughout the Software Development Lifecycle.

Developer Experience (DevEx)

Measures how effectively developers can use the Internal Developer Platform to deliver software.

Internal Developer Portal

A centralized, self-service interface where developers discover services, documentation, templates, and workflows required to build and operate applications.

Introducing Red Hat Developer Hub

The Challenge

Modern Software Factories bring together many capabilities, including source control, CI/CD, GitOps, development environments, documentation, observability, and security.

While these capabilities improve software delivery, they also introduce a new challenge for developers.

> If a Software Factory consists of many different tools and platform capabilities, how do developers efficiently discover and use them without learning every underlying technology?

Without a unified experience, developers often need to:

- Navigate multiple web consoles
- Search for project documentation
- Locate source code repositories and build pipelines
- Understand platform-specific implementation details
- Request infrastructure or platform services from different teams

Instead of building software, developers spend valuable time learning tools and navigating platform complexity.



As Software Factories grow, developer productivity depends not only on automation, but also on providing a consistent and discoverable developer

> experience.

The Internal Developer Portal

The solution is an **Internal Developer Portal**.

An Internal Developer Portal provides developers with a single, self-service interface for interacting with the Software Factory.

Rather than accessing separate applications for source control, CI/CD, deployment, documentation, observability, and development environments, developers can access everything from one centralized portal.

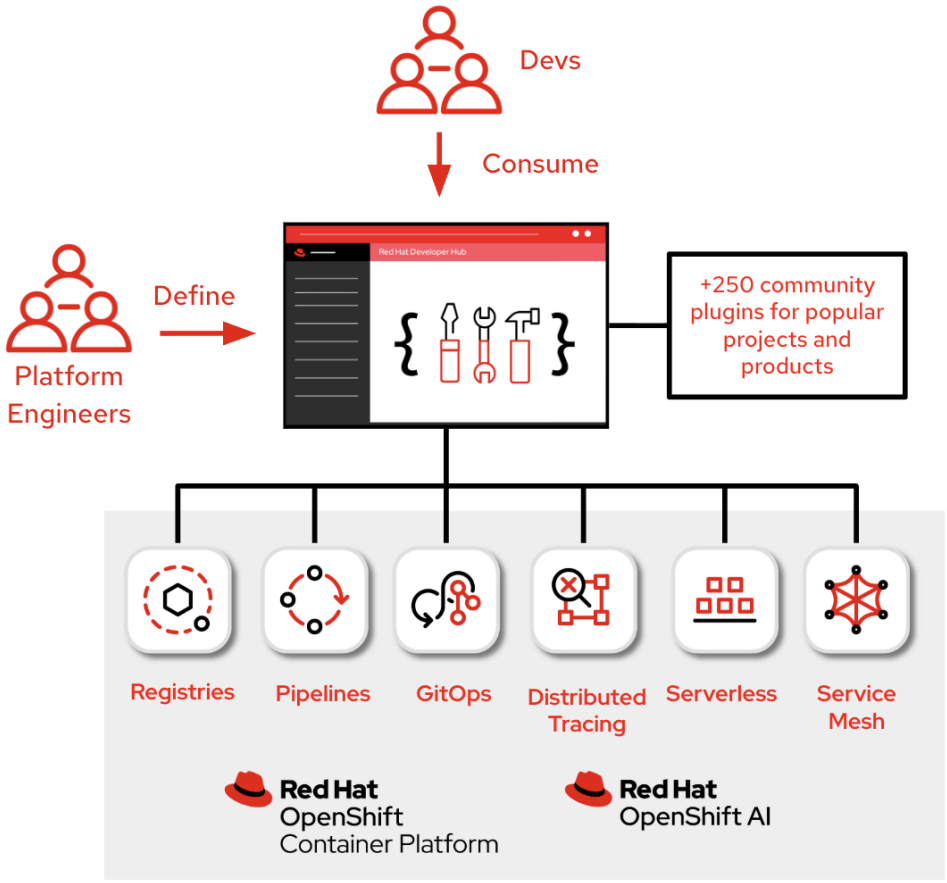


Figure 1. Internal Developer Portal within the Software Factory(Click the image to view it in full size in a new browser tab)

Platform Engineers build and maintain the Software Factory by integrating capabilities such as source control, CI/CD pipelines, GitOps, software catalogs, development environments, observability, and security services.

Developers simply consume these capabilities through the Internal Developer Portal using a consistent self-service experience.

What is Red Hat Developer Hub?

Red Hat Developer Hub (RHDH) is an enterprise-grade Internal Developer Portal built on the CNCF Backstage open source project.

Rather than replacing the tools already used within the Software Factory, Developer Hub integrates them into a single developer experience.

Developers can use Developer Hub to:

- Discover software components, APIs, services, and infrastructure resources
- Create new applications using standardized Software Templates (Golden Paths)
- Launch cloud-based development environments
- Monitor build pipelines and deployment status
- Access project documentation through TechDocs
- Navigate the complete application delivery lifecycle from a single interface

Instead of learning every underlying platform technology, developers interact with a consistent, self-service portal.

Developer Hub within the Software Factory

Developer Hub serves as the entry point into the Software Factory, bringing together platform capabilities into a unified workflow.

Software Factory Capability	**How Developer Hub Integrates**
Source Control	Creates and links application repositories.
Development Environment	Launches cloud-based workspaces using OpenShift Dev Spaces.
Continuous Integration	Displays build pipelines and execution status through Tekton integrations.
Continuous Delivery	Shows deployment status through GitOps integrations such as Argo CD.
Software Catalog	Provides a searchable inventory of software components, APIs, services, and ownership information.
Documentation	Makes project documentation available through TechDocs.
Observability	Integrates monitoring, tracing, and visualization platforms through plugins.

Rather than interacting with each platform independently, developers access these capabilities through one consistent interface.

Business Value

By introducing an Internal Developer Portal, organizations gain:

- A single entry point into the Software Factory
- Improved discoverability of software components and documentation
- Self-service application creation through Software Templates

- Standardized development workflows
- Reduced developer cognitive load
- Faster onboarding and improved developer productivity



Red Hat Developer Hub is not another development tool.

It provides the unified developer experience that enables teams to discover, consume, and interact with the capabilities of a Software Factory through a single self-service portal.

Reflection

▼ *Think about your own organization (Click to expand)*



- How many different tools do developers use during a typical workday?
- Where do developers go to find documentation or application ownership information?
- Which platform capabilities could benefit from self-service access?
- How would a single developer portal improve onboarding and daily development activities?

```
<div class="key-card">
```

Key Takeaway

Red Hat Developer Hub provides a unified, self-service entry point into the Software Factory, allowing developers to discover platform capabilities, create applications, and manage the software delivery lifecycle without needing to learn every underlying tool.

```
</div>
```

Lab 1: Install and configure RHDH

In this lab, you'll install Red Hat Developer Hub on OpenShift and deploy a minimal portal configuration that will serve as the foundation for the remaining exercises in this module.

By the end of this lab, you will have:

- Installed the Red Hat Developer Hub Operator
- Created a minimal Developer Hub application configuration
- Deployed a Red Hat Developer Hub instance
- Verified that you can access and sign in to the portal

Install the Red Hat Developer Hub Operator

1. Get the OpenShift web console URL from your RHDP lab environment page and open in the browser.
2. Sign in using the `admin` credentials provided on the RHDP Lab Environment page.
3. From the navigation menu, select **[Ecosystem]** → **[Software Catalog]**.

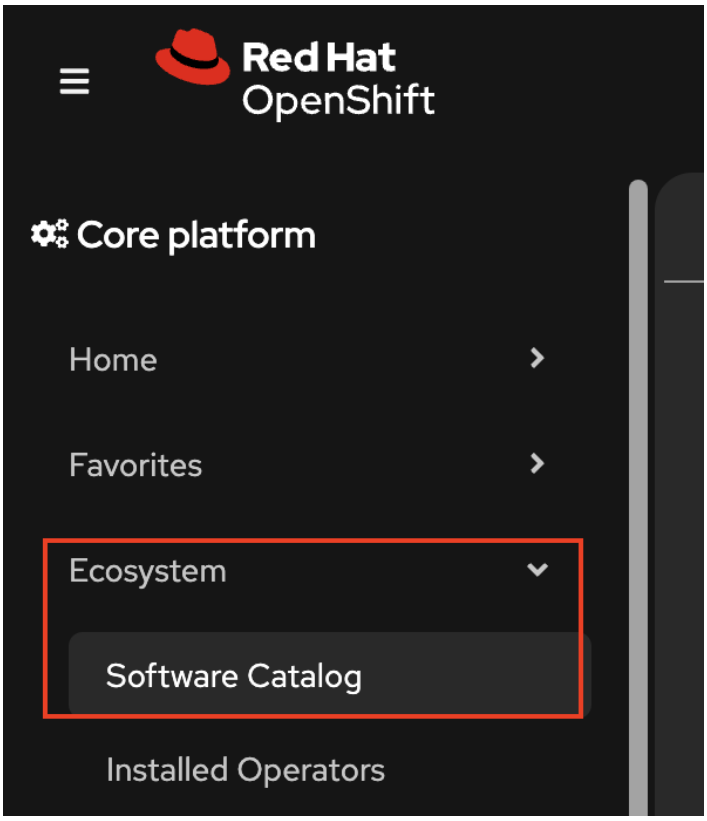


Figure 2. Ecosystem → Software Catalog(Click the image to view it in full size in a new browser tab)

4. Search for **Red Hat Developer Hub**.

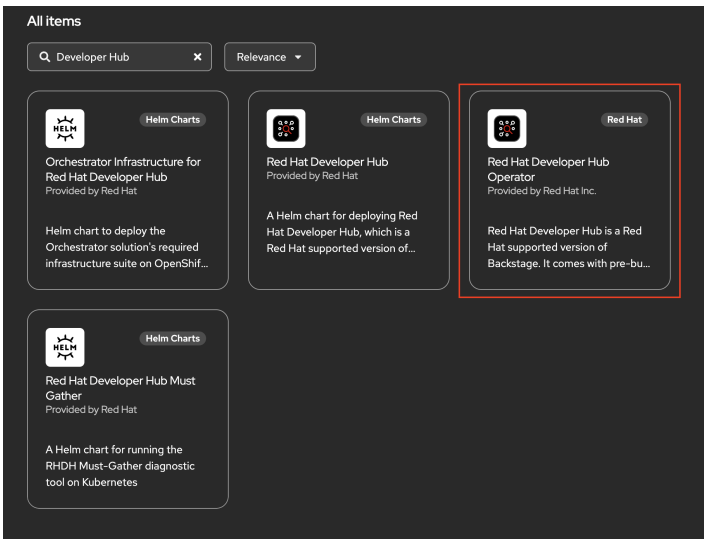


Figure 3. Red Hat Developer Hub Operator(Click the image to view it in full size in a new browser tab)



It may take a few moments for OperatorHub to finish loading all available operators.

5. Select the operator and click **[Install]**.

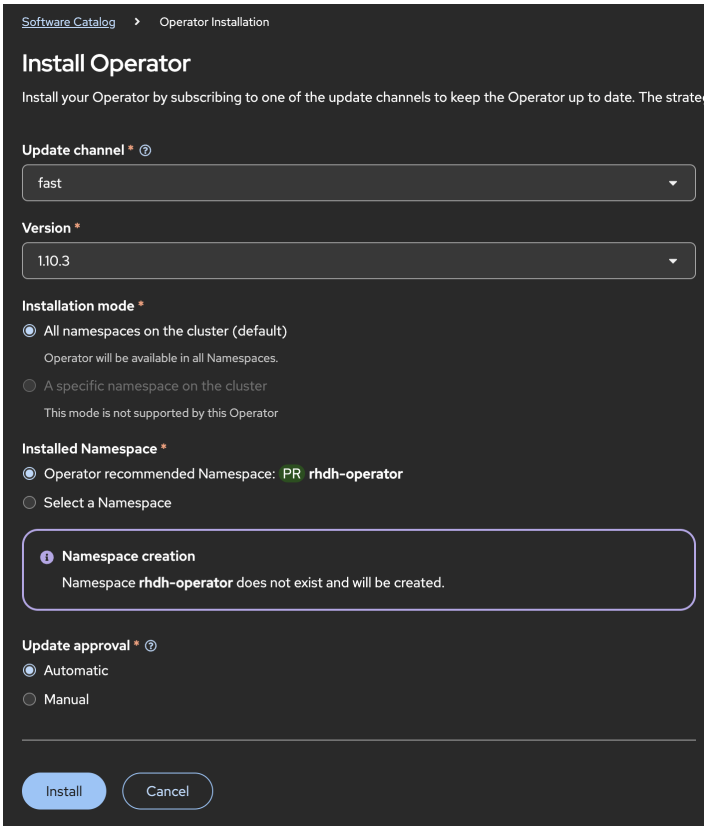


Figure 4. Install Red Hat OpenShift Hub Operator(Click the image to view it in full size in a new browser tab)

6. Accept the following installation settings:
   - **Installation namespace:** `rhdh-operator`
   - **Installation mode:** `All namespaces`
   - **Approval strategy:** `Automatic`
7. Click **[Install]**.
8. Wait until the operator status changes to **Succeeded**.

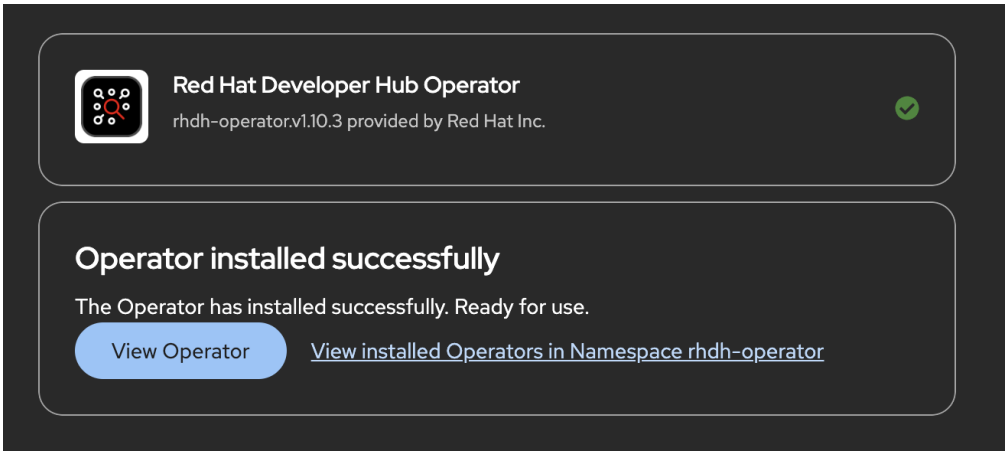


Figure 5. Red Hat OpenShift Developer Hub Installed(Click the image to view it in full size in a new browser tab)

Create the Developer Hub Project

Deploy Red Hat Developer Hub into a dedicated OpenShift project.

```
oc create namespace setup-rhdh
```

Verify the project was created successfully.

```
oc get namespace setup-rhdh
```

The output should include the `setup-rhdh` namespace.

Create the Application Configuration

Red Hat Developer Hub loads its application settings from an `app-config.yaml` file. For this training, you'll provide the configuration using a Kubernetes ConfigMap.

1. Copy the output of the following command. This value is the **openshift_cluster_ingress_domain** and is required in the next step.

   ```
   oc get ingresses.config.openshift.io cluster -o jsonpath='{.spec.domain}{"\n"}'
   ```

2. Apply the following manifest, replacing **{openshift_cluster_ingress_domain}** with the value obtained from the previous command.

   ```
   oc apply -f - <<'EOF'
   apiVersion: v1
   kind: ConfigMap
   metadata:
     name: rhdh-config
     namespace: setup-rhdh
   data:
     app-config.yaml: |
       app:
         title: Red Hat Developer Hub
       backend:
         reading:
           allow:
             - host: 'gitlab.{openshift_cluster_ingress_domain}'
       auth:
         providers:
           guest:
             dangerouslyAllowOutsideDevelopment: true
   EOF
   ```

   The `backend.reading.allow` setting allows Red Hat Developer Hub to retrieve

Software Catalog information and Software Templates from the GitLab instance used throughout this training.



Guest authentication (`dangerouslyAllowOutsideDevelopment`) is intended only for development and training environments. Production deployments should integrate with an enterprise identity provider such as Red Hat Build of Keycloak.

Verify the ConfigMap

Verify that the ConfigMap was created successfully.

```
oc get configmap rhdh-config -n setup-rhdh
```

The output should include the `rhdh-config` ConfigMap.

Deploy Red Hat Developer Hub

The Red Hat Developer Hub Operator watches for `Backstage` custom resources and automatically provisions the application, database, routes, and supporting Kubernetes resources.

Apply the following manifest.

```
oc apply -f - <<'EOF'
apiVersion: rhdh.redhat.com/v1alpha5
kind: Backstage
metadata:
  name: rhdh
  namespace: setup-rhdh
spec:
  application:
    appConfig:
      mountPath: /opt/app-root/src
      configMaps:
        - name: rhdh-config
    route:
      enabled: true
  database:
    enableLocalDb: true
EOF
```

Verify the Deployment

Wait for the Developer Hub pods to become available.

```
oc get pods -n setup-rhdh -w
```

Verify that both pods are in the `Running` state, it might take couple of minutes.

The output should include pods similar to:

- `backstage-rhdh-xxxxxxxx`
- `postgresql-xxxxxxxx`

Next, verify that the Backstage custom resource was created.

```
oc get backstage -n setup-rhdh
```

Access Red Hat Developer Hub

Retrieve the Developer Hub route and open the route URL in your browser:

```
oc get route backstage-rhdh \
  -n setup-rhdh \
  -o jsonpath='https://{.spec.host}{"\n"}'
```

If you receive an **Application is not available** message, wait a minute for the pods to finish starting and refresh the page.

On the sign-in screen, select **Guest** and click **[Enter]**.

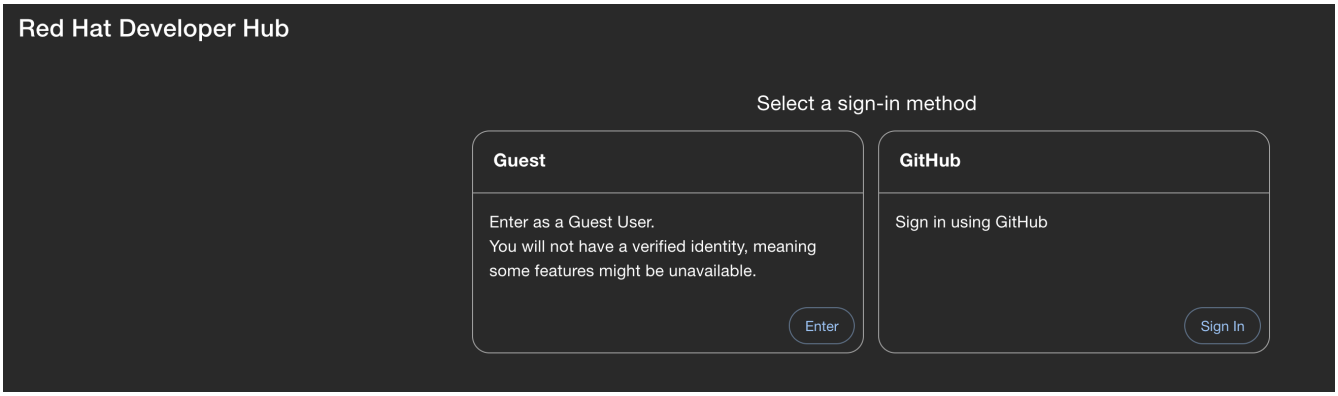


Figure 6. Sample Sign-in Screen (Click the image to view it in full size in a new browser tab)

After signing in, you should see the **Red Hat Developer Hub** home page.

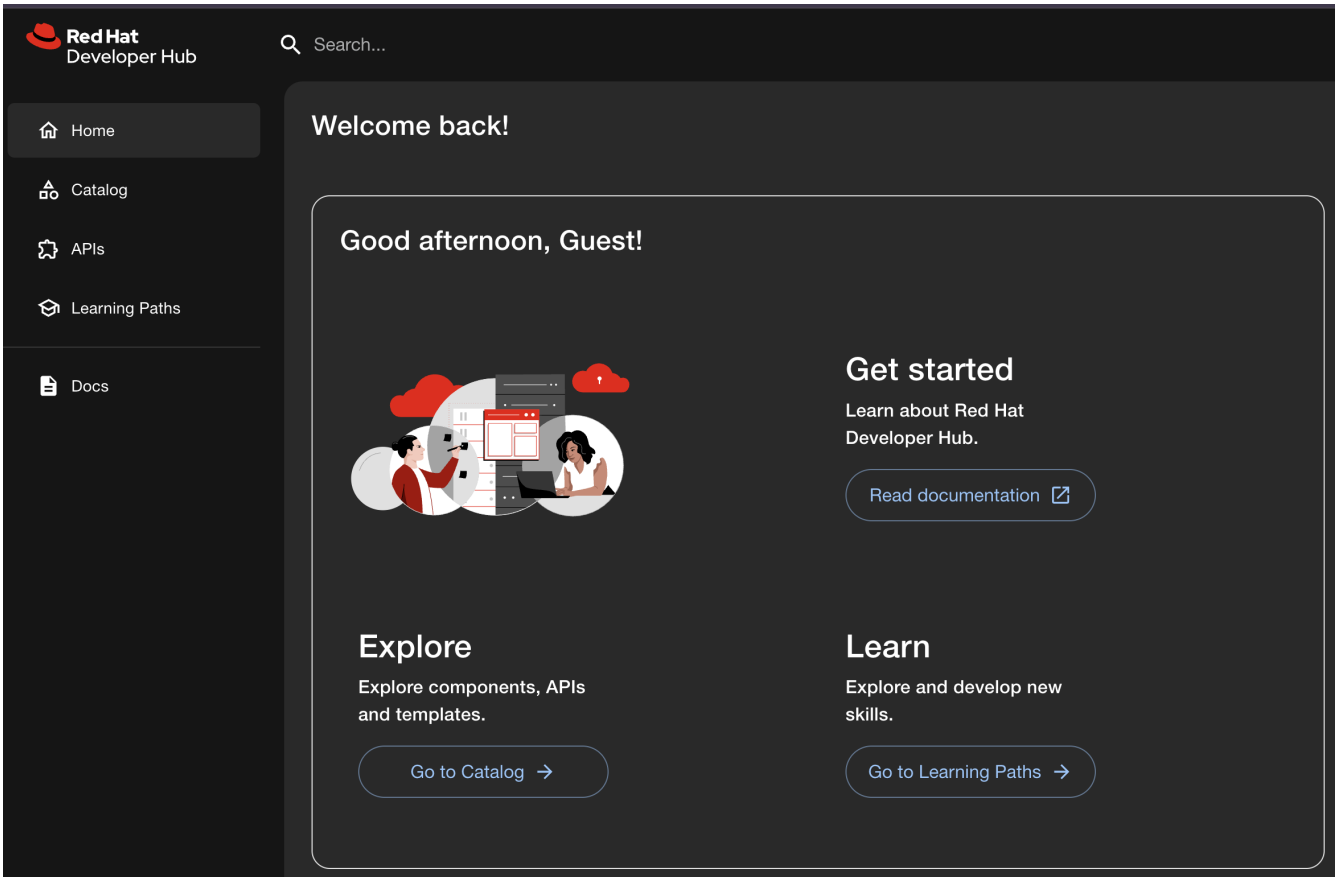


Figure 7. RHDH Home Page (Click the image to view it in full size in a new browser tab)



Guest authentication is used only for this self-paced training to simplify setup. In production environments, organizations typically authenticate users through enterprise identity providers such as Red Hat Build of Keycloak.

In the next lab, you'll begin customizing Red Hat Developer Hub by importing Software Templates and integrating it with the Software Factory services deployed earlier.

Software Templates in RHDH



What's in this section?

In this section, you'll learn how Red Hat Developer Hub uses Software Templates (Golden Paths) to provide self-service application creation.

You'll explore how templates automate project setup, provision repositories and CI/CD resources, understand the Software Template workflow, and use the Software Catalog to discover and manage software components.

By the end of this section, you'll understand how Software Templates and the Software Catalog reduce developer cognitive load, standardize application delivery, and enable consistent self-service workflows across the Software Factory.

Self-Service with Software Templates

The Challenge

Creating a new application often involves much more than writing code.

Developers frequently need to create repositories, configure CI/CD pipelines, prepare deployment manifests, register applications in software catalogs, and configure development environments before they can begin implementing business functionality.

These repetitive setup activities consume valuable development time and often lead to inconsistent project structures across teams.



When every team creates new projects manually, onboarding becomes slower, engineering practices become inconsistent, and maintaining organizational standards becomes increasingly difficult.

What are Software Templates?

Software Templates, also known as **Golden Paths**, provide standardized project blueprints that automate the creation of new applications.

Rather than starting from an empty repository, developers begin with an organization-approved template that already includes the required project structure, configurations, and platform

integrations.

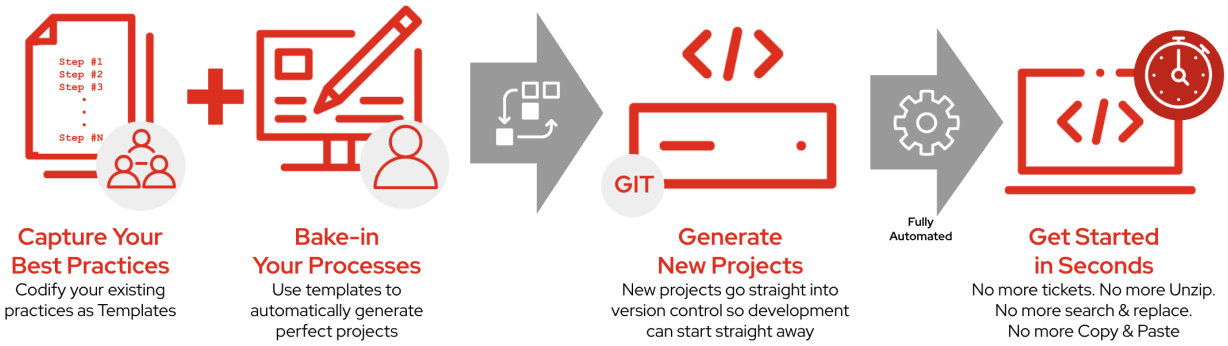


Figure 8. Self-Service Application Creation(Click the image to view it in full size in a new browser tab)

Platform Engineers capture organizational best practices and package them into reusable Software Templates.

Developers simply provide a few project-specific details, while the platform automatically provisions the remaining resources.

How Software Templates Work

Software Templates automate the activities required to bootstrap a new application.

A template can provision:

- Project structure and source code scaffold
- Git repository creation
- CI/CD pipeline configuration
- Deployment manifests, such as Helm charts or Kustomize overlays
- OpenShift Dev Spaces workspace configuration
- Software Catalog registration
- Organizational standards and recommended configurations

Instead of performing these tasks manually, developers complete a simple self-service form, and the platform generates a production-ready project automatically.

Self-Service Project Creation

When a developer selects a Software Template in Red Hat Developer Hub, the platform executes the template and provisions the required resources.

This includes creating repositories, generating source code, configuring pipelines, preparing deployment resources, and registering the application within the Software Factory.

The generated project is immediately available for development, allowing developers to focus on implementing business functionality rather than configuring platform services.

Business Value

By adopting Software Templates, organizations gain:

- Faster application onboarding
- Standardized project structures across development teams
- Self-service application creation
- Consistent security and organizational best practices
- Reduced manual project setup
- Improved developer productivity



Software Templates do not replace developer creativity.

They automate the repetitive foundation of every project, allowing developers to focus on building business functionality while Platform Engineers continuously improve the underlying templates and organizational standards.

Reflection

▼ *Think about your own organization (Click to expand)*



- How is a new application created in your organization today?
- Which project setup activities are still performed manually?
- Which organizational standards could be automated through reusable templates?
- How would self-service project creation improve developer productivity?

```
<div class="key-card">
```

Key Takeaway

Software Templates enable self-service application creation by automating project setup, embedding organizational best practices, and providing developers with a production-ready foundation so they can focus on building business functionality.

```
</div>
```

Software Template Workflow

Behind the Scenes of a Software Template

Creating a new application from Red Hat Developer Hub appears simple.

A developer selects a **Software Template**, provides a few project details, and clicks **Create**.

Behind that simple interface, however, Developer Hub orchestrates a series of automated tasks that prepare the application for development, testing, deployment, and operations.

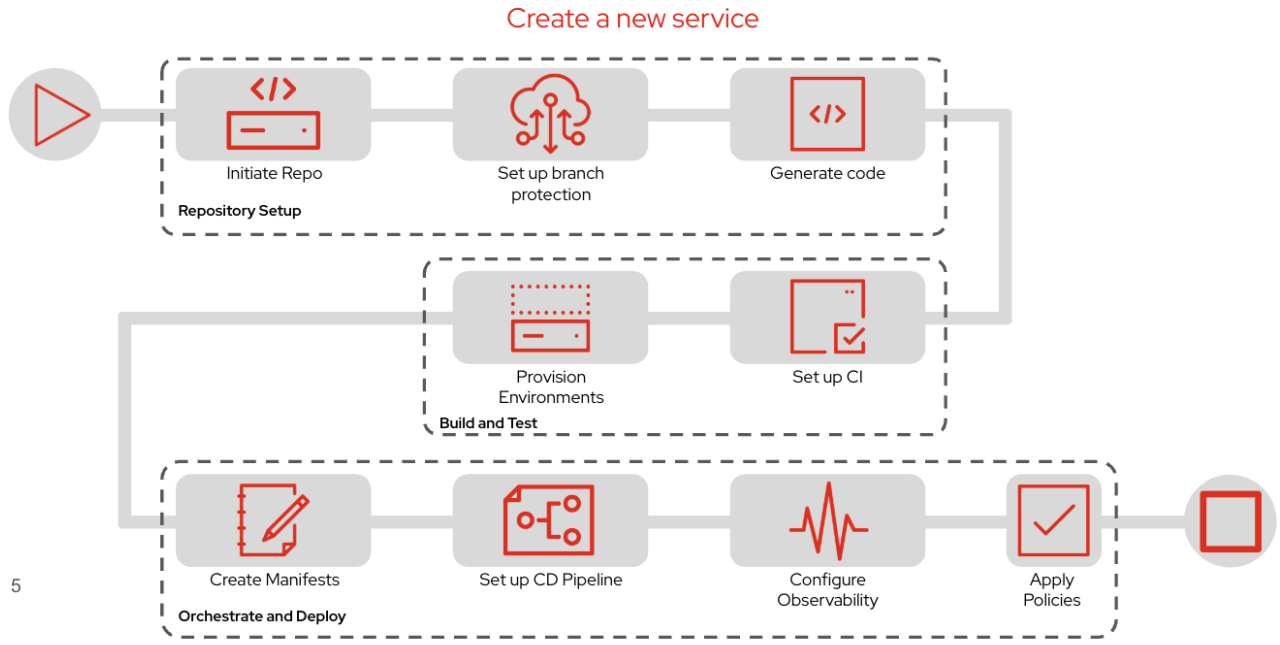


Figure 9. Software Template Workflow (Click the image to view it in full size in a new browser tab)

Rather than generating only application source code, the Software Template establishes the complete foundation required for the application to participate in the organization's Software Factory.

Repository Provisioning

The first stage prepares the application's source repository.

Developer Hub automatically performs tasks such as:

- Creating a new Git repository
- Configuring repository permissions
- Applying branch protection policies
- Generating the initial project structure from the selected template

Instead of manually creating repositories and copying starter projects, developers receive a project that already follows organizational standards.

Build and Continuous Integration

Next, the template prepares everything required for Continuous Integration (CI).

Typical automation includes:

- Creating build pipelines
- Configuring automated builds
- Preparing testing workflows
- Integrating with the organization's CI platform

Once developers commit their code, the application is immediately ready to be built, tested, and validated through automated pipelines.

Deployment and Operations

The final stage prepares the application for deployment into Kubernetes.

The template automatically generates resources such as:

- Kubernetes deployment manifests
- Helm charts or Kustomize overlays
- GitOps deployment configuration
- Observability integrations
- Security and governance policies

By the time the template finishes, the application is ready to progress through the Software Factory using standardized deployment workflows.

End-to-End Automation

Software Templates automate much more than project creation.

Every generated application already includes the capabilities needed throughout the software delivery lifecycle, including:

- Source control
- Continuous Integration
- Continuous Delivery
- Kubernetes deployment resources
- Observability configuration
- Security and governance

Developers can immediately begin implementing business functionality instead of spending time assembling platform infrastructure.



Software Templates are maintained by Platform Engineers.

When organizational standards evolve, Platform Engineers update the template

> once. Every new application generated from that template automatically inherits the latest best practices without requiring developers to manually reconfigure their projects.

Reflection

▼ *Think about your own organization (Click to expand)*



- How much time do developers spend creating repositories and configuring new projects?
- Which setup activities are still performed manually?
- Which of these tasks could be automated using Software Templates?
- How would standardized project generation improve consistency across development teams?

Key Takeaway

A **Software Template** automates far more than project scaffolding. It provisions repositories, CI/CD pipelines, deployment resources, and platform integrations, allowing developers to focus on building applications while the Software Factory handles the surrounding infrastructure.

Software Template Structure

Templates are defined using YAML, but are rendered as a rich form in the Red Hat Developer Hub UI when used by development teams.

Understand the Template Structure

A Software Template contains two major parts:

- **template.yaml** — describes the workflow executed by Developer Hub
- **skeleton/** — contains the files copied into newly generated applications

The repository structure will look similar to:

```
golden-path-quarkus/
 ├──── template.yaml
 └──── skeleton/
     ├──── devfile.yaml
     ├──── pom.xml
     ├──── src/
     ├──── catalog-info.yaml
     ├──── tekton/
```

```
└──── gitops/
```

Everything inside the skeleton directory becomes the starting point for every project created from this template.

Software Catalog

The Challenge

As organizations adopt microservices and Platform Engineering, the number of software assets grows rapidly.

Instead of a single application, modern systems often consist of dozens or even hundreds of services, APIs, databases, libraries, and infrastructure resources owned by different teams.

Without a centralized inventory, developers often struggle to answer questions such as:

- Which team owns this service?
- Where is the source code located?
- Which APIs does this application expose?
- Is this application still actively maintained?
- Where can I find its documentation?

Searching across multiple tools wastes valuable development time and increases cognitive load.



When software ownership and documentation are scattered across different systems, developers spend more time searching for information than building new features.

What is the Software Catalog?

The **Software Catalog** provides a centralized inventory of software assets across the organization.

Rather than maintaining spreadsheets or separate documentation portals, Platform Engineers register software assets in the catalog, allowing developers to discover applications, services, APIs, documentation, and ownership information from a single location.

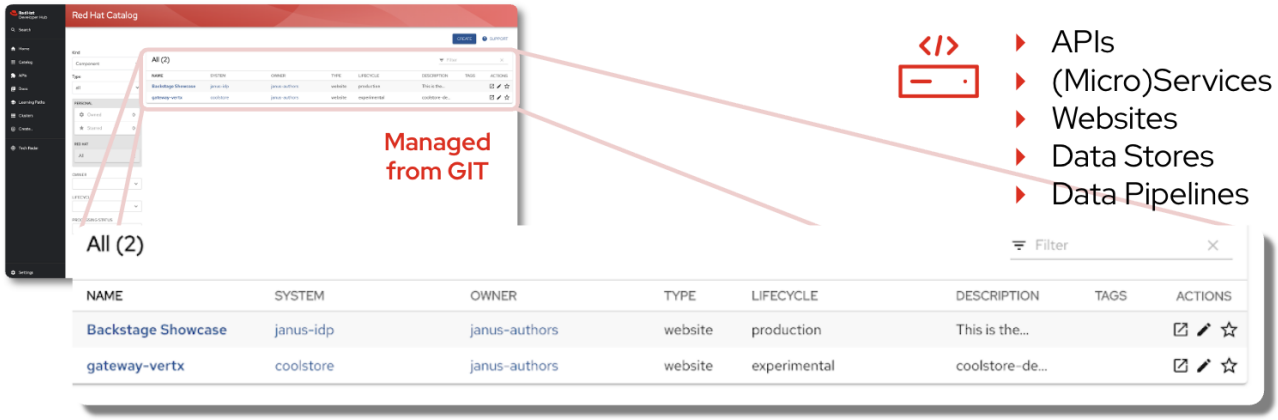


Figure 10. Software Catalog (Click the image to view it in full size in a new browser tab)

The Software Catalog becomes the single source of truth for software components and the relationships between them, making it easier for developers to understand how applications fit within the broader Software Factory.

What Does the Software Catalog Track?

The Software Catalog organizes software into several types of entities.

Entity Type	Description
Components	Individual software components such as microservices, web applications, backend services, frontend applications, or shared libraries.
APIs	REST, GraphQL, gRPC, and other APIs exposed by applications.
Resources	Infrastructure resources such as databases, object storage, message brokers, or Kubernetes resources.
Systems	Collections of related components that together deliver a business application or platform.
Domains	Higher-level business or organizational groupings that contain multiple systems.
Users	Individual developers or engineers associated with software components.
Groups	Development teams responsible for owning and maintaining software.

More Than an Inventory

The Software Catalog provides much more than a list of applications.

Each catalog entry includes valuable metadata that helps developers understand and manage software throughout its lifecycle, including:

- Application owner
- Source code repository
- Lifecycle stage

- System relationships
- API dependencies
- Technical documentation
- CI/CD pipelines
- Deployment status

Instead of searching multiple tools, developers can quickly understand an application's purpose, ownership, and operational status from a single page.

Managed as Code

One of the key principles of Red Hat Developer Hub is that the Software Catalog is **managed as code**.

Each application contains a `catalog-info.yaml` file in its source repository. This file defines metadata such as:

- Component name
- Owner
- Lifecycle
- Type
- Relationships to other software assets

Because this metadata lives alongside the application's source code, updates follow the same Git-based review and approval process used for application development.

This ensures that the catalog remains accurate, version-controlled, and automatically synchronized with the latest state of the application.

Business Value

The Software Catalog benefits both developers and Platform Engineers.

For developers:

- Quickly discover applications, services, and APIs.
- Identify application owners and responsible teams.
- Access repositories, documentation, and deployment information from one location.
- Understand relationships between software components.

For Platform Engineers:

- Maintain an accurate inventory of software assets.
- Improve governance and software ownership.
- Standardize metadata across development teams.

- Enable self-service discovery throughout the organization.



Think of the **Software Catalog** as the organization's software inventory.

Just as a library catalog helps readers quickly find books, the Software Catalog helps developers discover applications, services, APIs, documentation, ownership information, and platform resources from a single interface.

Reflection

▼ *Think about your own organization (Click to expand)*



- How do developers discover existing applications and services today?
- Can you easily identify who owns a particular application or API?
- Where is technical documentation stored?
- How much time is spent searching across different tools for this information?

```
<div class="key-card">
```

Key Takeaway

A **Software Catalog** serves as the central inventory of the Software Factory. It enables developers to discover software assets, understand ownership and relationships, and access documentation, repositories, and operational information through a single self-service portal.

```
</div>
```

Lab 2: RHDH Templates

In this lab, you'll create and register a **Software Template (Golden Path)** in Red Hat Developer Hub.

Instead of manually creating repositories, pipelines, deployment manifests, and catalog entries, you'll define a reusable template that automates project creation for developers.

By the end of this lab, you will have:

- Created a Git repository for a Software Template
- Defined a Backstage `template.yaml`
- Added a reusable application skeleton
- Registered the template in Red Hat Developer Hub
- Verified the template appears in the **Create** page

Create the Template Repository

Software Templates are stored in Git repositories, just like application source code.

Create a new repository that will contain the Golden Path template.

1. Retrive GitLab URL and navigate to the GitLab account.

```
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"
echo $GITLAB_URL
```

2. Navigate to the **Software Factory** group.

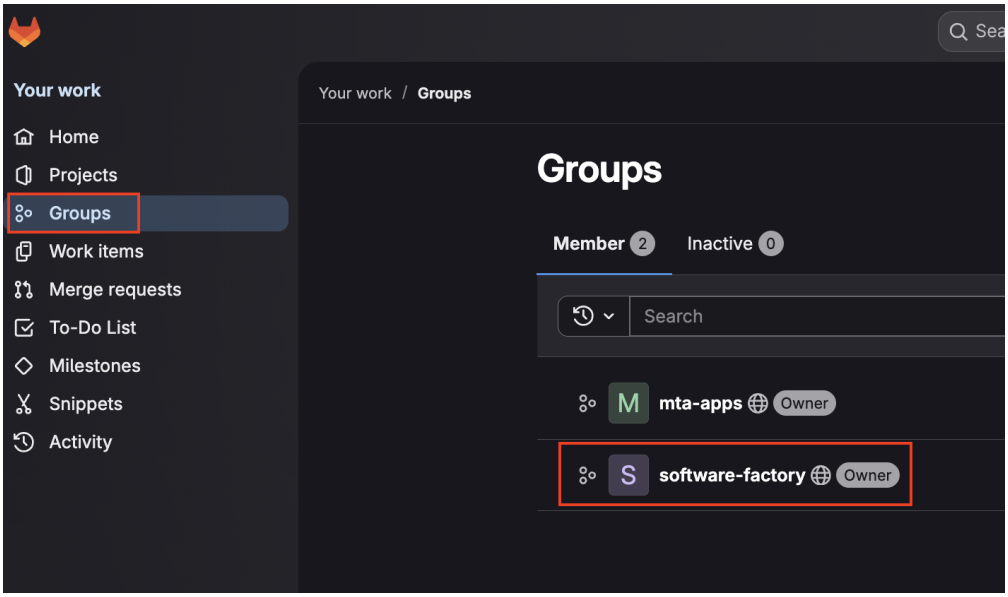


Figure 11. Software Factory Group(Click the image to view it in full size in a new browser tab)

3. Click **[Create Project]**.

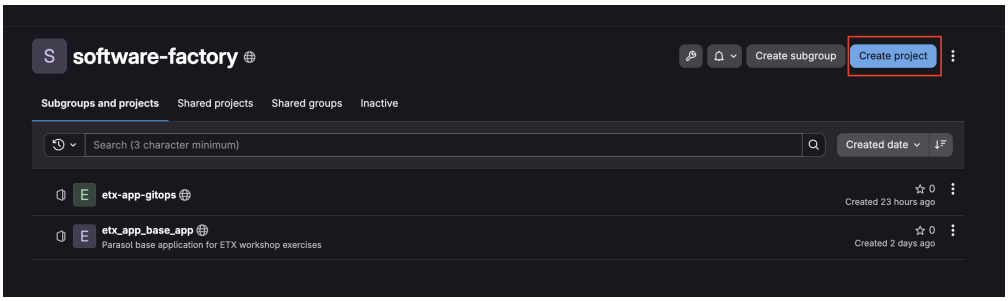


Figure 12. Create Project(Click the image to view it in full size in a new browser tab)

4. Click **[Create Blank Project]**

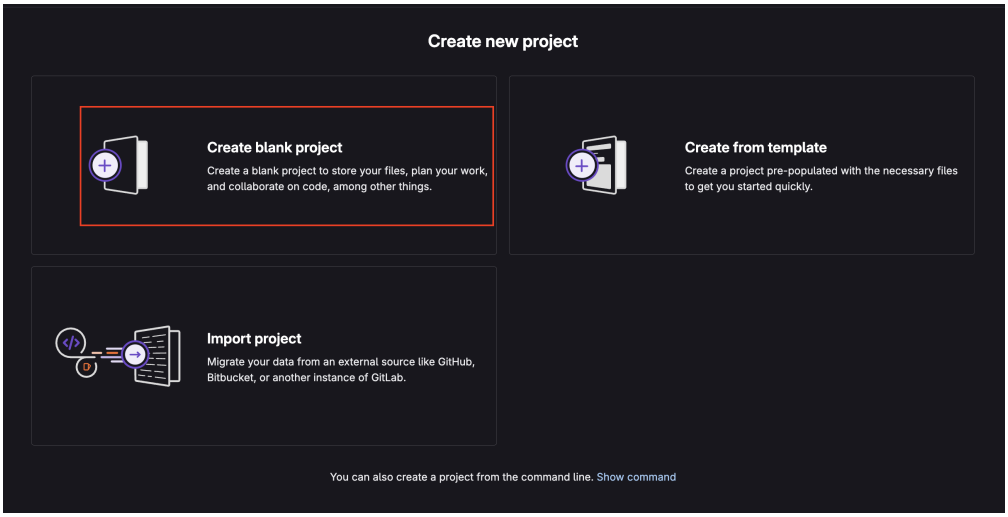


Figure 13. Create Blank Project(Click the image to view it in full size in a new browser tab)

5. Create a new **Public** project named `golden-path-quarkus`. Leave **Initialize repository with a README** unchecked.

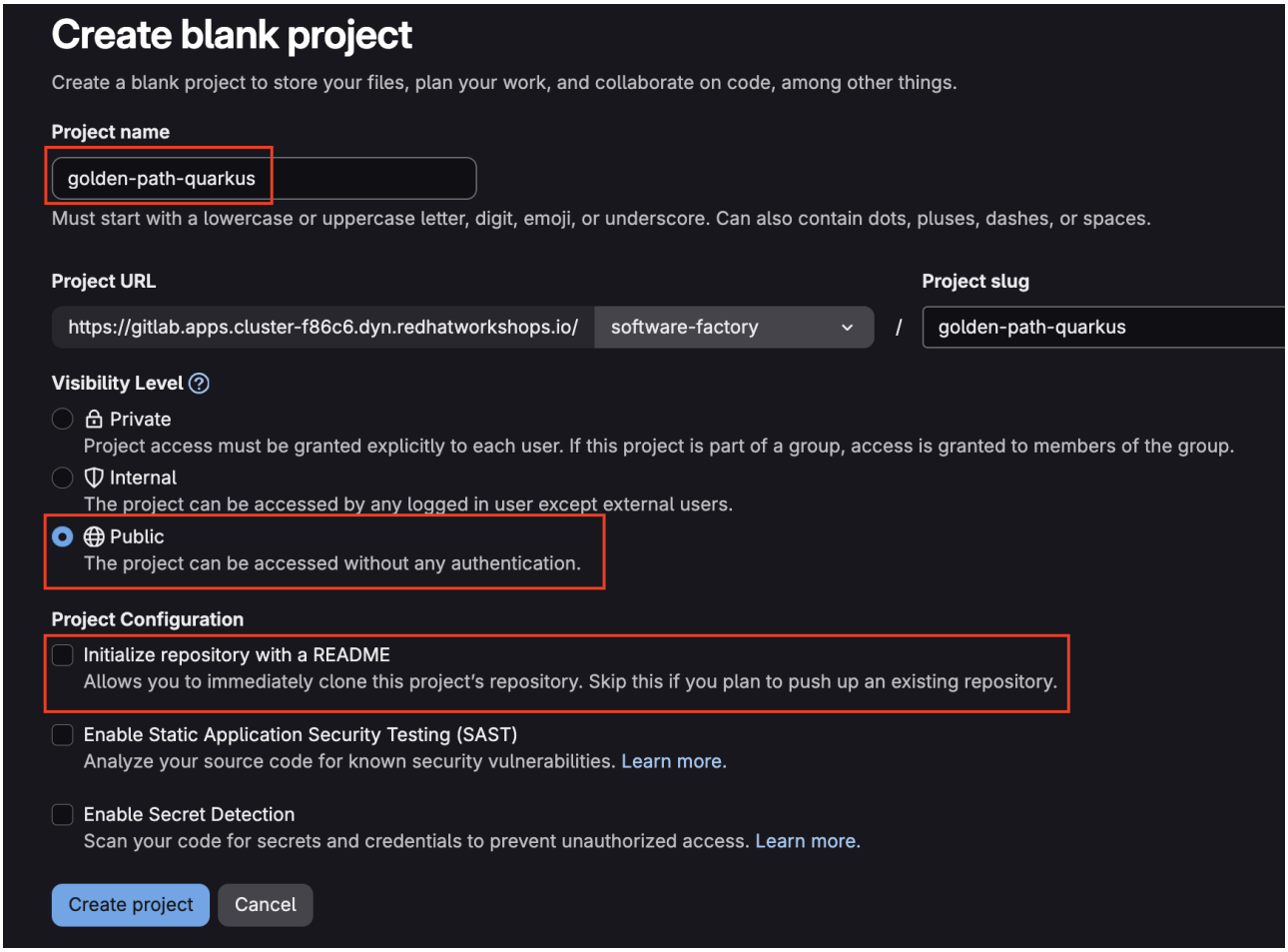


Figure 14. GitLab Project(Click the image to view it in full size in a new browser tab)

6. Clone the repository locally.

```
git clone ${GITLAB_URL}/software-factory/golden-path-quarkus.git
cd golden-path-quarkus
```

Create the Template Definition

Create a file named `template.yaml` in the repository root.

Before creating the template, retrieve your cluster's unique identifier (GUID) which is needed in the template configuration.

1. Get your cluster domain and store it in a variable:

   ```
   CLUSTER_DOMAIN=$(oc get ingresses.config.openshift.io cluster -o
   jsonpath='{.spec.domain}' | sed 's/apps\.//')
   echo "Your cluster domain: ${CLUSTER_DOMAIN}"
   ```

   The output will be in the format `cluster-XXXXX.dyn.redhatworkshops.io` where `XXXXX` is your cluster's unique GUID (for example: `cluster-lhpfm.dyn.redhatworkshops.io`).

2. Create the template definition:

   ```
   cat << 'EOF' > template.yaml
   apiVersion: scaffolder.backstage.io/v1beta3
   kind: Template
   metadata:
     name: quarkus-golden-path
     title: Quarkus Application (Golden Path)
     description: |
       Create a Quarkus application with CI/CD pipeline, GitOps deployment,
       Dev Spaces workspace, and OpenTelemetry instrumentation pre-configured.
     tags:
       - quarkus
       - java
       - recommended
   spec:
     owner: platform-team
     type: service
     parameters:
       - title: Application Details
         required:
           - name
           - owner
         properties:
           name:
             title: Application Name
             type: string
             pattern: "^[a-z0-9-]+$"
           description:
             title: Description
             type: string
           owner:
             title: Owner
   ```

```
          type: string
          ui:field: OwnerPicker
        port:
          title: Port
          type: number
          default: 8080
          description: Override the port exposed for the application
        groupId:
          title: Group ID
          type: string
          default: io.janus
          description: Maven Group ID eg (io.janus)
        artifactId:
          title: Artifact ID
          type: string
          default: quarkusapp
          description: Maven Artifact ID
        javaPackageName:
          title: Java Package Namespace
          type: string
          default: io/janus/quarkusapp
          description: Name for the Java Package (ensure to use the / character as
this is used for folder structure) should match Group ID and Artifact ID

    - title: Repository
      required:
        - repoUrl
      properties:
        repoUrl:
          title: Repository Location
          type: string
          ui:field: RepoUrlPicker
          ui:options:
            allowedHosts:
              - gitlab.apps.cluster-f86c6.dyn.redhatworkshops.io ①
            allowedOwners:
              - software-factory
  steps:
    - id: fetch-skeleton
      name: Fetch Skeleton
      action: fetch:template
      input:
        url: ./skeleton
        values:
          name: ${{ parameters.name }}
          description: ${{ parameters.description }}
          owner: ${{ parameters.owner }}
          applicationType: api
          port: ${{ parameters.port }}
          groupId: ${{ parameters.groupId }}
          artifactId: ${{ parameters.artifactId }}
```

```
          javaPackageName: ${{ parameters.javaPackageName }}
          repoCloneUrl: https://${{ (parameters.repoUrl | parseRepoUrl).host }}/${{
(parameters.repoUrl | parseRepoUrl).owner }}/${{ (parameters.repoUrl |
parseRepoUrl).repo }}.git

    - id: publish
      name: Publish to GitLab
      action: publish:gitlab
      input:
        repoUrl: ${{ parameters.repoUrl }}
        settings:
          description: ${{ parameters.description }}
        defaultBranch: main
        repoVisibility: public

    - id: register
      name: Register in Catalog
      action: catalog:register
      input:
        repoContentsUrl: ${{ steps['publish'].output.repoContentsUrl }}
        catalogInfoPath: /catalog-info.yaml

  output:
    links:
      - title: Repository
        url: ${{ steps['publish'].output.remoteUrl }}
      - title: Open in Catalog
        icon: catalog
        entityRef: ${{ steps['register'].output.entityRef }}
EOF
```

① This line contains the GitLab hostname that must match your cluster. The example value `cluster-f86c6.dyn.redhatworkshops.io` is a placeholder.

3. Substitute the placeholder cluster domain with your actual cluster domain:

```
sed -i "s/cluster-f86c6.dyn.redhatworkshops.io/gitlab.apps.${CLUSTER_DOMAIN}/g"
template.yaml
```

Verify the substitution was successful:

```
grep "allowedHosts:" -A 1 template.yaml
```

The output should show your actual cluster domain, for example:

```
allowedHosts:
  - gitlab.apps.cluster-lhpfm.dyn.redhatworkshops.io
```

This template performs three automated tasks:

- Generates a project from the skeleton
- Publishes the generated project to GitLab
- Registers the new application in the Software Catalog

Copy the Skeleton

Use the below command to use preconfigured skeleton folder:

```
curl -L \
  "https://codeload.github.com/redhat-gpte-devopsautomation/software-
templates/tar.gz/main" \
  | tar -xz --strip-components=3 \
    software-templates-main/scaffolder-templates/quarkus-backend-template/skeleton
```

Create the Catalog Entity

Copy the output of the following command. This value is the **openshift_cluster_ingress_domain** and is required in the next step.

```
oc get ingresses.config.openshift.io cluster -o jsonpath='{.spec.domain}{"\n"}'
```

Inside the `skeleton` directory, create a file named `catalog-info.yaml`. Execute the following manifest, replacing **{openshift_cluster_ingress_domain}** with the value obtained from the previous command.

```
cat << 'EOF' > skeleton/catalog-info.yaml
apiVersion: backstage.io/v1alpha1
kind: Component
metadata:
  name: ${{ values.name }}
  description: ${{ values.description }}
  annotations:
    backstage.io/techdocs-ref: dir:.
    argocd/app-name: ${{ values.name }}-dev
    janus-idp.io/tekton: ${{ values.name }}
    che.eclipse.org/che-server: https://devspaces.{openshift_cluster_ingress_domain}
    app.openshift.io/vcs-uri: ${{values.repoCloneUrl}}
  links:
    - url:
https://devspaces.{openshift_cluster_ingress_domain}/#${{values.repoCloneUrl}}
      title: OpenShift Dev Spaces
spec:
  type: service
  lifecycle: production
  owner: ${{ values.owner }}
```

```
EOF
```

Every application generated from this template will automatically include this catalog metadata.

This allows Red Hat Developer Hub to understand:

- Application ownership
- Lifecycle
- Documentation
- CI/CD integrations
- Deployment relationships

Create the Dev File

Add devfile.yaml to the skeleton directory:

```
cat << 'EOF' > skeleton/devfile.yaml
schemaVersion: 2.2.0
metadata:
  name: ${{ values.name }}
  version: 1.0.0
  displayName: Quarkus Java Application
  description: Development environment for standard and native Quarkus Java apps.
  tags:
    - Java
    - Quarkus
  projectType: Quarkus
  language: Java

components:
  # Main development container toolset
  - name: tools
    container:
      image: registry.redhat.io/devspaces/udi-rhel9:latest
      memoryLimit: 2Gi
      memoryRequest: 512Mi
      cpuLimit: '2'
      cpuRequest: '500m'
      mountSources: true
      env:
        - name: QUARKUS_HTTP_HOST
          value: 0.0.0.0
      endpoints:
        - name: http
          targetPort: ${{ values.port }}
          exposure: public
          protocol: http
        - name: dev-ui
```

```
          targetPort: ${{ values.port }}
          path: /q/dev-ui
          exposure: public
          protocol: http
        - name: debug
          targetPort: 5005
          exposure: none
          protocol: tcp
      volumeMounts:
        - name: m2-cache
          path: /home/user/.m2

  # Persistent local cache volume for Maven dependencies
  - name: m2-cache
    volume:
      size: 3Gi

commands:
  # 1. Start Quarkus Live-Coding Dev Mode
  - id: dev
    exec:
      label: "Start Quarkus Dev Mode"
      component: tools
      workingDir: ${PROJECTS_ROOT}
      commandLine: "./mvnw quarkus:dev"
      group:
        kind: run
        isDefault: true

  # 2. Package standard JVM JAR file
  - id: package-jvm
    exec:
      label: "Package JVM JAR"
      component: tools
      workingDir: ${PROJECTS_ROOT}
      commandLine: "./mvnw clean package -DskipTests=true"
      group:
        kind: build

  # 3. Package Native Executable (Requires GraalVM or explicit container build flags)
  - id: package-native
    exec:
      label: "Package Native Binary"
      component: tools
      workingDir: ${PROJECTS_ROOT}
      commandLine: "./mvnw clean package -Pnative -DskipTests=true"
      group:
        kind: build

  # 4. Run Unit and Integration Tests
  - id: test
```

```
    exec:
      label: "Run Tests"
      component: tools
      workingDir: ${PROJECTS_ROOT}
      commandLine: "./mvnw test"
      group:
        kind: test
EOF
```

Commit and Push the Template

Commit the new template repository.

```
git add .
git commit -m "feat: initial Golden Path template"
```

Configure Git authentication using your Personal Access Token from Module 3:

```
# Replace <YOUR-TOKEN> with the GitLab Personal Access Token created in Module 3, Lab
7
git remote set-url origin https://root:<YOUR-TOKEN>@${GITLAB_URL#https://}/software-
factory/golden-path-quarkus.git
```

Push the template to GitLab:

```
git push origin main
```



The Personal Access Token (starting with `glpat-`) was created in **Module 3, Section 2, Lab 7: Create the GitOps Repository**. If you don't have this token, you'll need to create a new one in GitLab by navigating to your avatar → Edit profile → Access Tokens.

Software Templates Integrations



What's in this section?

In this section, you'll learn how to register Software Templates in Red Hat Developer Hub and integrate them with the Software Factory.

You'll explore how Developer Hub templates use integrations and plugins to automate application creation, register software components, and connect with platform services. You'll then register a Golden Path template and configure OpenShift OAuth to provide secure enterprise authentication.

By the end of this section, you'll have a registered Software Template and a securely authenticated Developer Hub environment ready for self-service application creation.

Lab 3: Register the Template in Developer Hub

In this lab, you'll register a **Software Template (Golden Path)** in Red Hat Developer Hub by configuring a catalog location. You will update the existing Developer Hub configuration and verify that the Quarkus Golden Path is available in the Software Catalog.

By the end of this lab, you will have:

- Configured a catalog location for the Software Template
- Updated the Developer Hub configuration
- Verified that the Quarkus Golden Path is registered
- Reviewed the template metadata and input form

Configure the Software Template Catalog Location

Red Hat Developer Hub discovers Software Templates through catalog locations configured in app-config.yaml.

You'll update the existing rhdh-config ConfigMap created in the previous lab to register the Quarkus Golden Path template.

1. Navigate to the OpenShift web console and switch to the **setup-rhdh** project.

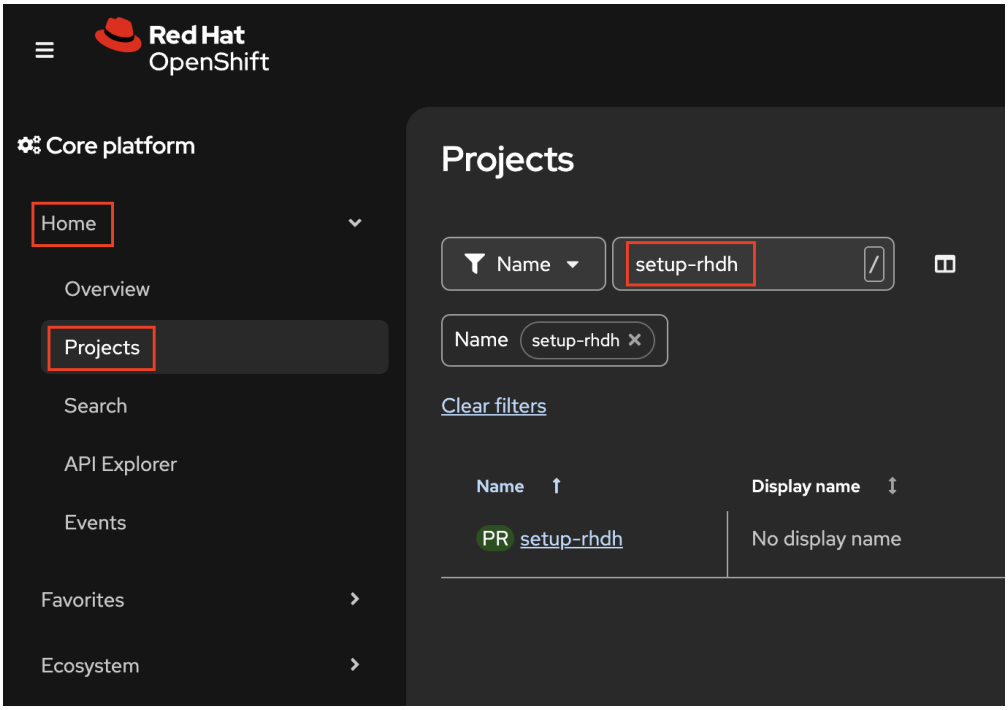


Figure 15. OpenShift Console (Click the image to view it in full size in a new browser tab)

2. From the navigation menu, select Workloads → ConfigMaps.

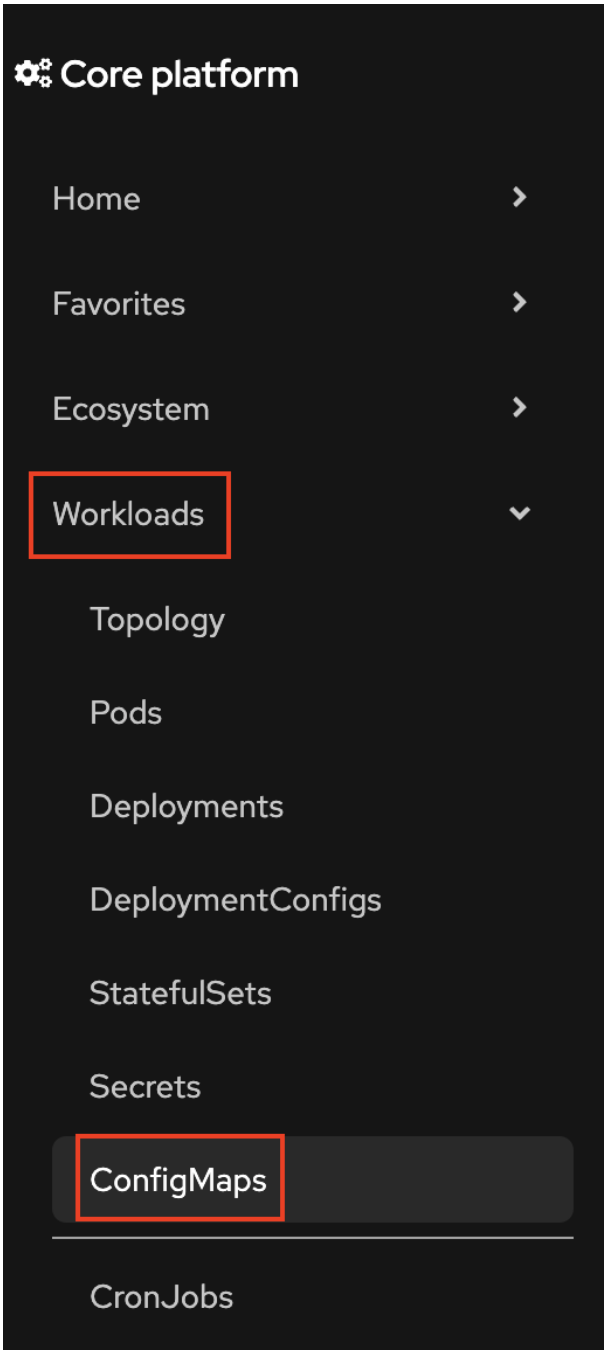


Figure 16. OpenShift Console Workloads (Click the image to view it in full size in a new browser tab)

3. Select rhdh-config.

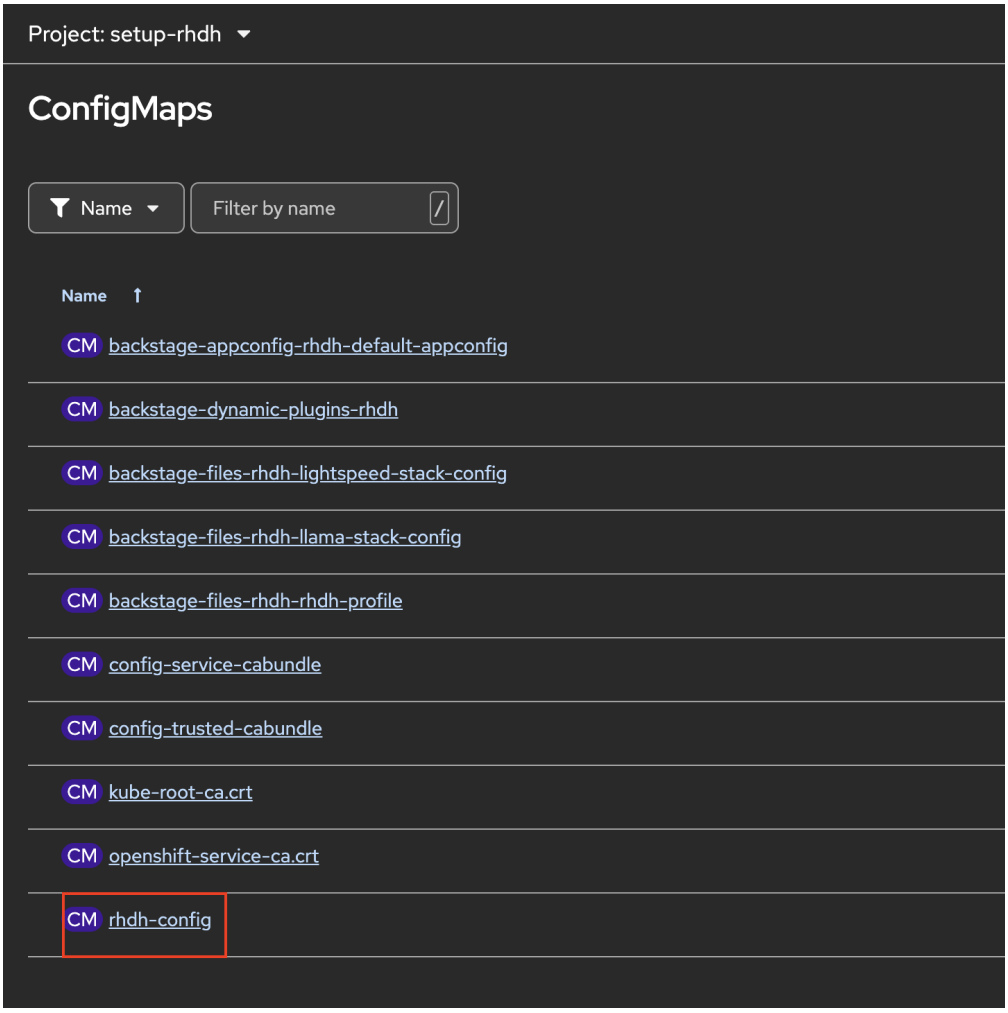


Figure 17. rhdh-config ConfigMap (Click the image to view it in full size in a new browser tab)

4. Select **[Actions]** → **[Edit ConfigMap]**.

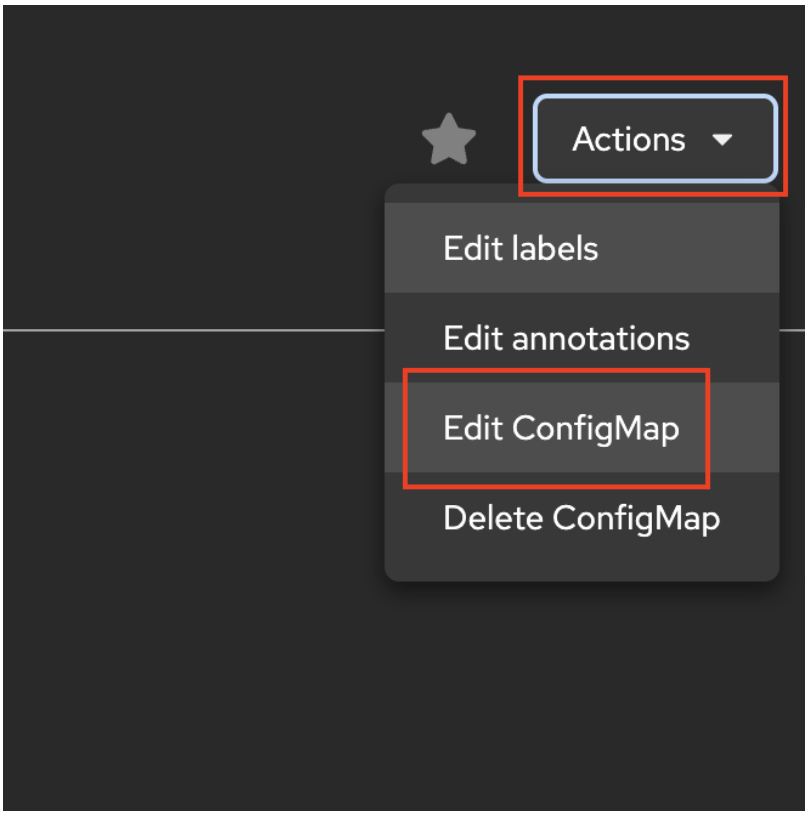


Figure 18. Actions (Click the image to view it in full size in a new browser tab)

5. Retrive GitLab URL and navigate to the GitLab account.

```
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"
echo $GITLAB_URL
```

6. Add the following `catalog` section under `data.app-config.yaml` and make sure to substitute ${GITLAB_URL} value with above command output.

```
catalog:
  locations:
    - type: url
      target: '${GITLAB_URL}/software-factory/golden-path-quarkus/-
/raw/main/template.yaml'
      rules:
        - allow: [Template]
```



Use `/-/raw/` in the URL path, not `/-/blob/`. RHDH needs to fetch the raw YAML content, not the GitLab web UI HTML page.

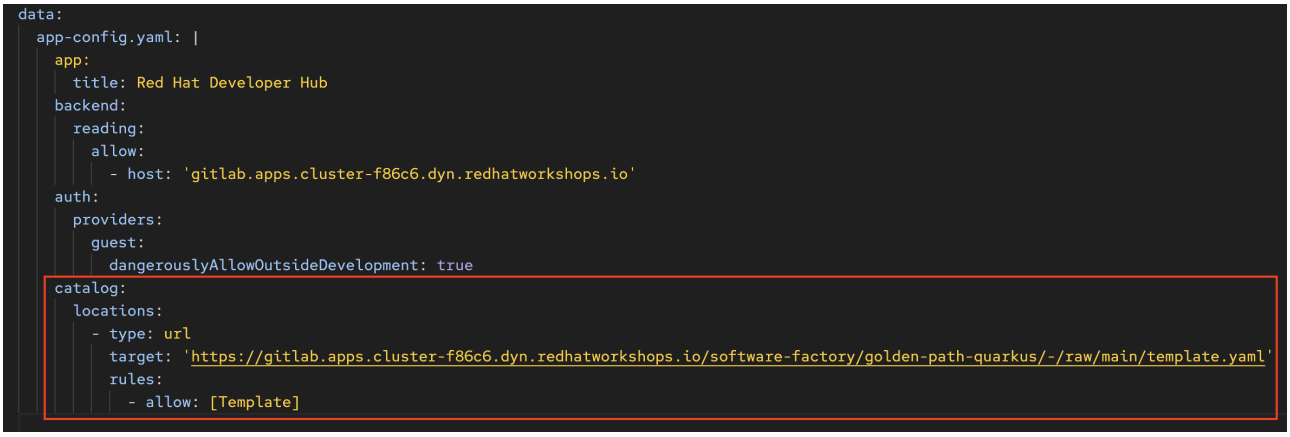


Figure 19. Sample setting(Click the image to view it in full size in a new browser tab)

Save the ConfigMap.



Do not replace the existing configuration. Add the template catalog location to the existing app-config.yaml settings.

Verify the Template

Verify that the Developer Hub application has restarted successfully after the configuration change.

Wait until the Developer Hub pod returns to the **Running** state.

```
oc get pods -n setup-rhdh -w
```

Retrieve the Developer Hub route and open the route URL in your browser:

```
oc get route backstage-rhdh \
  -n setup-rhdh \
  -o jsonpath='https://{.spec.host}{"\n"}'
```

In Developer Hub, click **[Catalog]**.

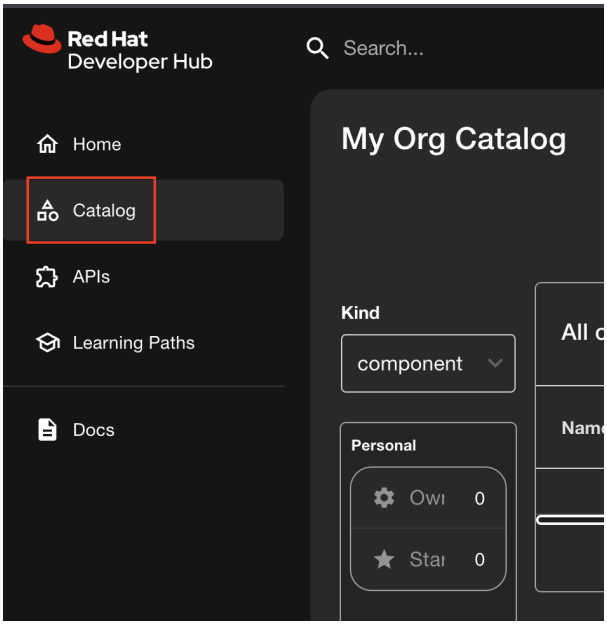


Figure 20. Select Catalog (Click the image to view it in full size in a new browser tab)

Select click **[Kind]** → click **[Template]**, you should now see the **Quarkus Golden Path** template.

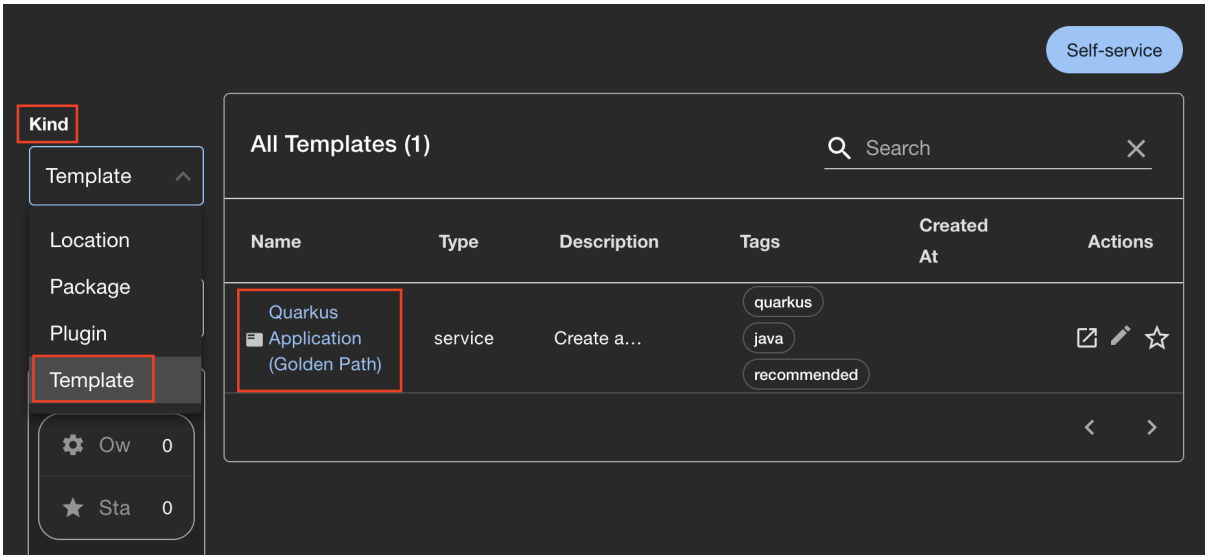


Figure 21. Quarkus Golden Path Template(Click the image to view it in full size in a new browser tab)

Select the template to verify its metadata and input form are displayed correctly.

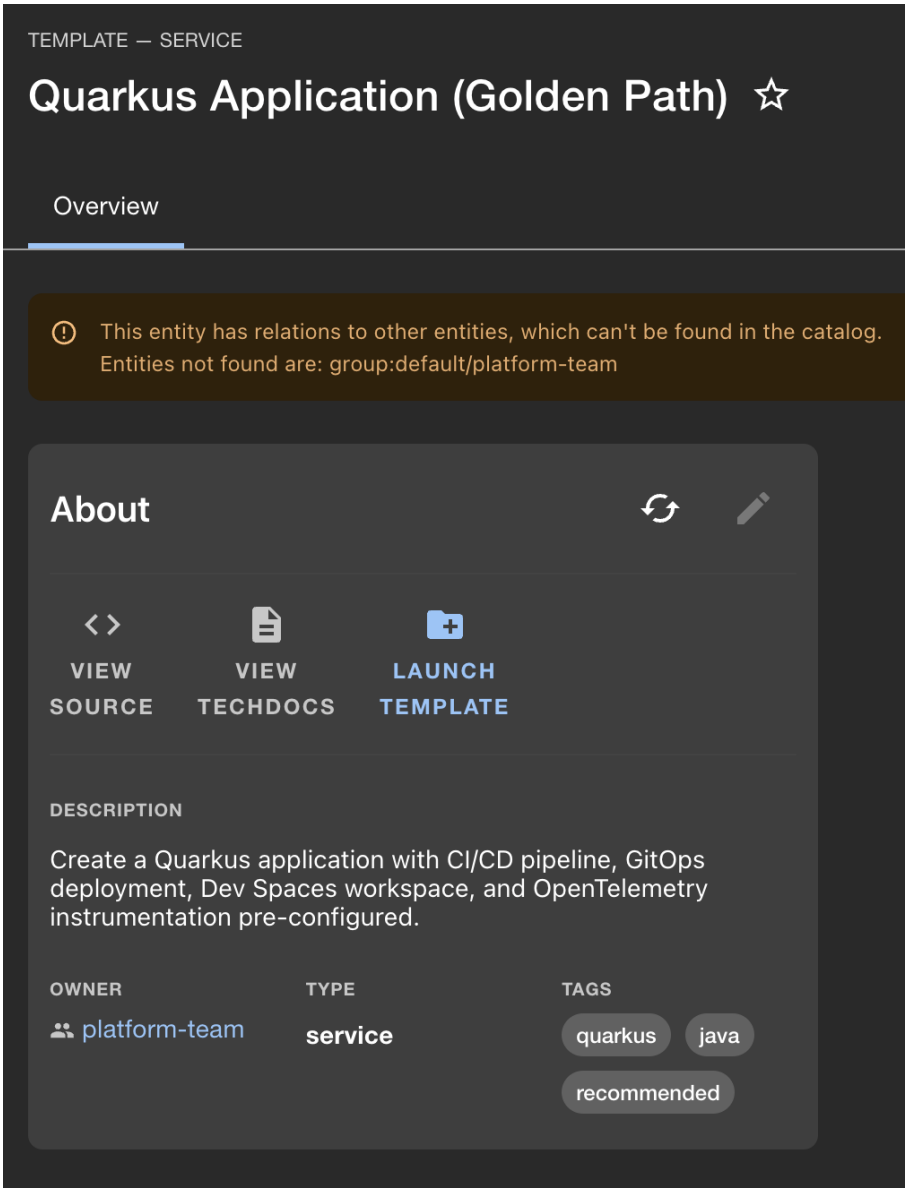


Figure 22. Template details(Click the image to view it in full size in a new browser tab)



Registering a Software Template makes the Golden Path discoverable to developers through the Developer Hub Software Catalog. Developers can then use the template to create applications through a standardized self-service workflow.

Lab 4: Configure OpenShift OAuth Authentication

In this lab, you'll replace the temporary Guest authentication used during installation with OpenShift OAuth authentication.

This allows developers to sign in using their OpenShift identities, providing a production-like authentication experience for Red Hat Developer Hub.

By the end of this lab, you will have:

- Created an OpenShift OAuth client
- Configured Developer Hub to authenticate using OpenShift OAuth
- Mounted authentication secrets into the Backstage application

- Verified user sign-in with OpenShift credentials
- Tested the Golden Path template using an authenticated user

Setup GitLab Integration

Create a Person Access Token(PAT) from GitLab to be used in RHDH. In production you would use a service account but for the purposes of this lab we can use a PAT.

Retrive GitLab URL and navigate to your **User settings** by opening below command's URL in browser.

```
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"
echo $GITLAB_URL/-/profile/preferences
```

Click **[Access]** → **[Personal access tokens]**

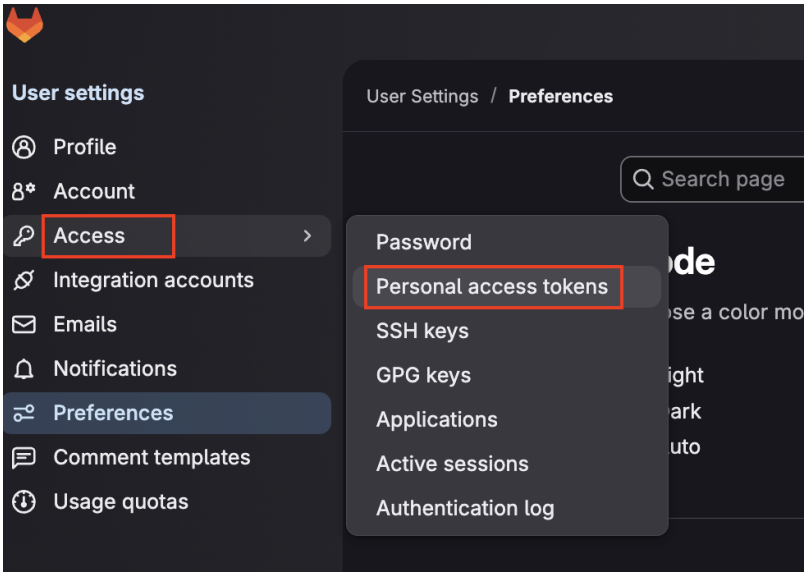


Figure 23. Actions(Click the image to view it in full size in a new browser tab)

Click **[Add new token]**

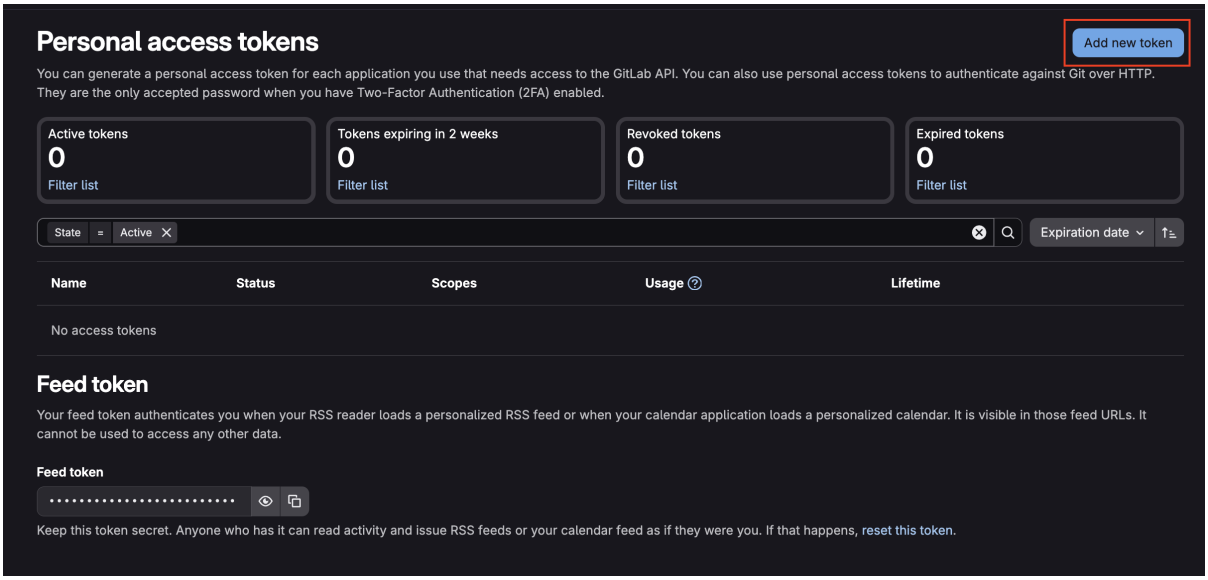


Figure 24. Add new token(Click the image to view it in full size in a new browser tab)

Give it a name and select the api scope and click **Generate Token**

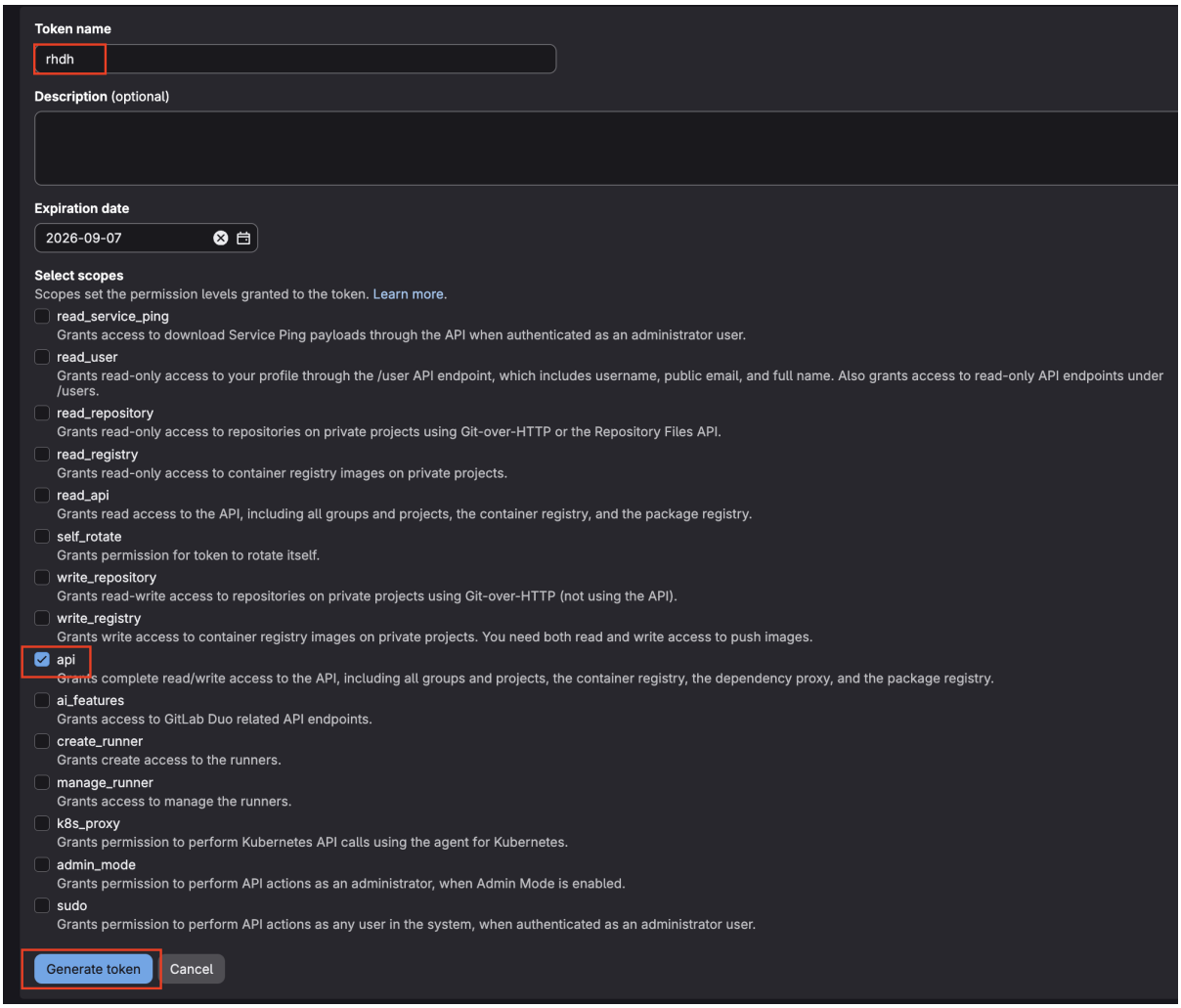


Figure 25. Token Settings(Click the image to view it in full size in a new browser tab)

You will see a Green box that says Your token, copy that token

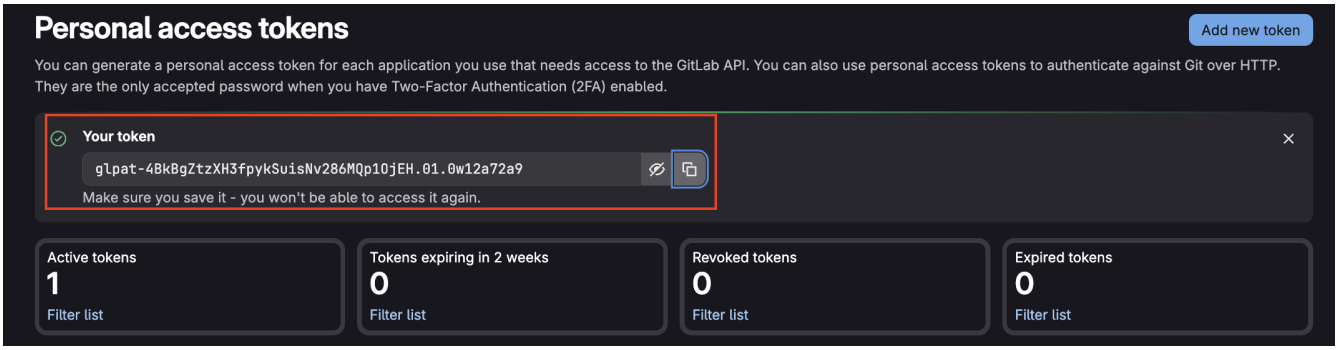


Figure 26. Token(Click the image to view it in full size in a new browser tab)

Run the following replacing {YOUR_PAT_HERE} with the **PAT** you copied:

```
export GITLAB_PAT={YOUR_PAT_HERE}
```

We will use the **GITLAB_PATH** environment variable in a later step when we setup the RHDH auth secret

Configure Keycloak OAuth Authentication

Replace guest authentication with Keycloak OAuth for production-like access.

Create the Auth Secret

RHDH will need to sign/encrypt the session cookie used during the OIDC flow. We need to generate and define a session secret. It can be any string but for real use it should be high entropy, e.g. ~32 random bytes

```
export SESSION_SECRET=$(python3 -c 'import secrets; print(secrets.token_hex(32))')
export GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"
export CLUSTER_DOMAIN=$(oc get ingresses.config.openshift.io cluster -o
jsonpath='{.spec.domain}{"\n"}')

oc create secret generic rhdh-auth-secret -n setup-rhdh \
  --from-literal=KEYCLOAK_RHDH_REALM=etx-lab \
  --from-literal=KEYCLOAK_CLIENT_ID=rhdh \
  --from-literal=KEYCLOAK_CLIENT_SECRET=etx-lab-workshop-gitlab-secret \
  --from-literal=KEYCLOAK_BASE_URL=https://etx-sso.${CLUSTER_DOMAIN} \
  --from-literal=SESSION_SECRET=${SESSION_SECRET} \
  --from-literal=GITLAB_TOKEN=${GITLAB_PAT}
```

Update the Developer Hub Configuration

The initial installation used the Guest authentication provider. Update the existing `app-config.yaml` to use OpenShift OAuth instead.

1. Navigate to your Openshift console, and switch to the **setup-rhdh** project.

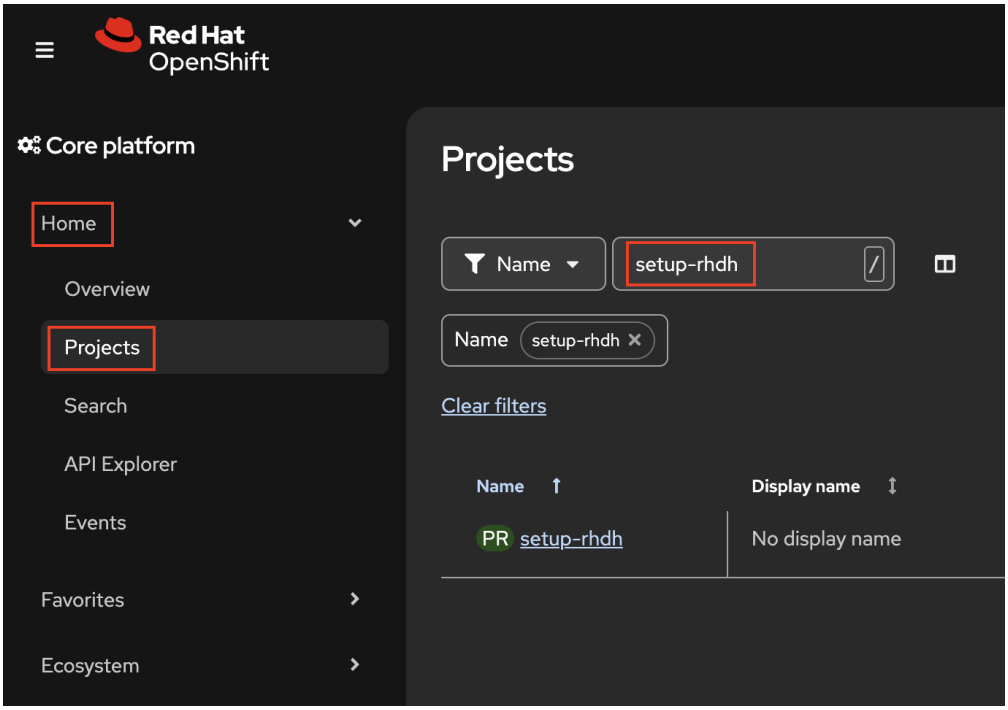


Figure 27. Openshift Console(Click the image to view it in full size in a new browser tab)

2. Open **Workloads** → **ConfigMaps**.

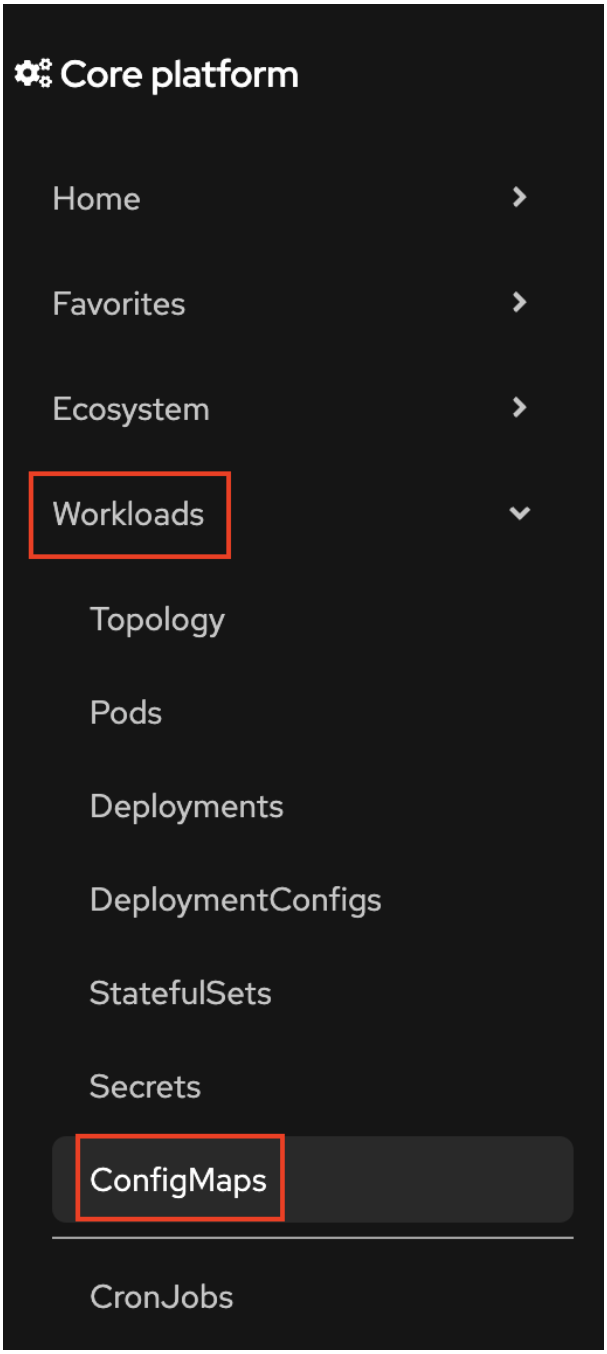


Figure 28. Openshift Console Workloads(Click the image to view it in full size in a new browser tab)

3. Select **rhdh-config**.

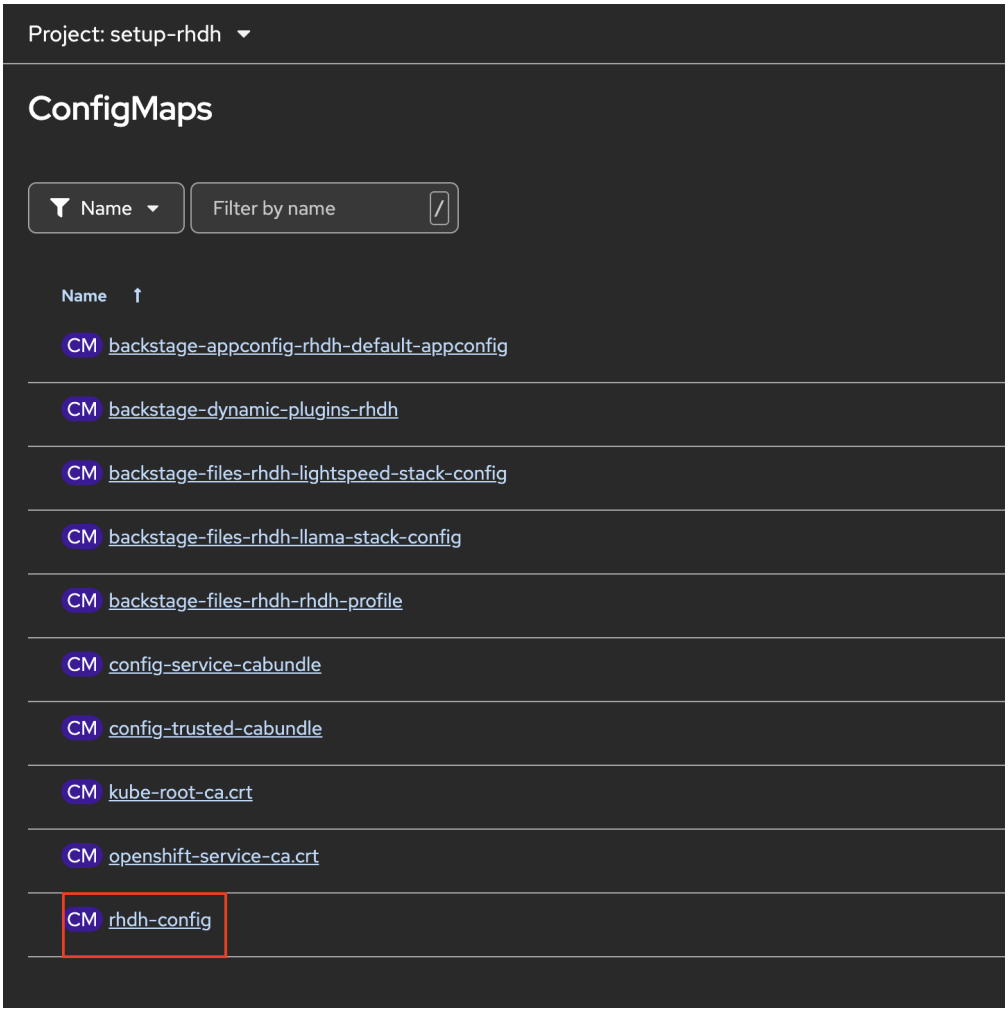


Figure 29. rhdh-config ConfigMap(Click the image to view it in full size in a new browser tab)

4. Choose **Actions** → **Edit ConfigMap**.

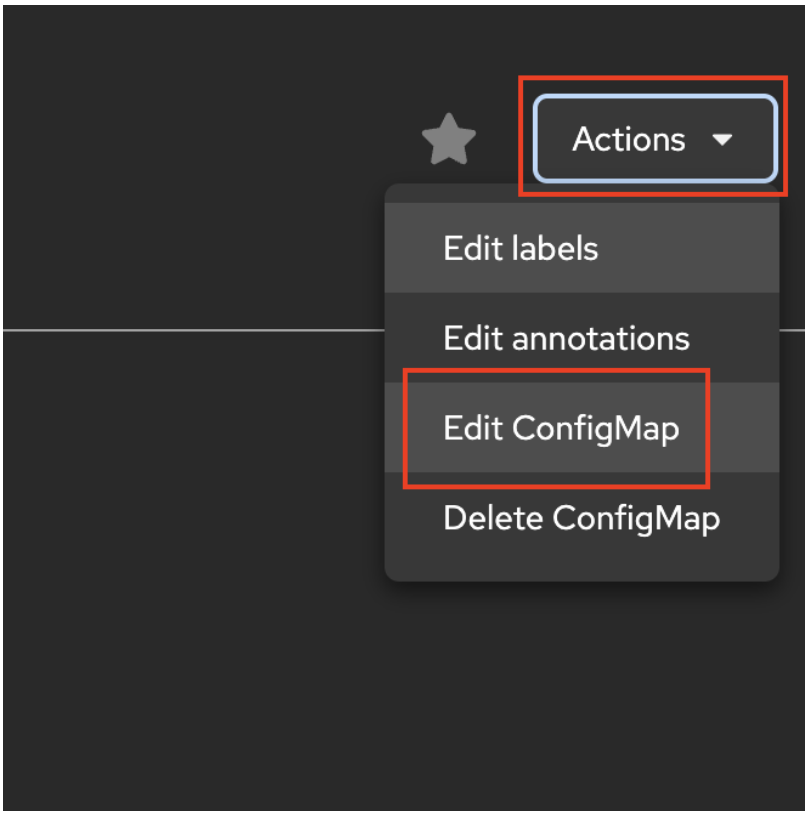


Figure 30. Actions(Click the image to view it in full size in a new browser tab)

5. Copy the output of the following command. This value is the **openshift_cluster_ingress_domain** and is required in the next step.

```
openshift_cluster_ingress_domain=$(oc get ingresses.config.openshift.io cluster -o
jsonpath='{.spec.domain}{"\n"}')
echo ${openshift_cluster_ingress_domain}
```

6. Replace the **Guest** authentication configuration with the following OIDC configuration, make sure to substitute **openshift_cluster_ingress_domain** with output receive from above command.

```
    signInPage: oidc
    auth:
      environment: production
      session:
        secret: ${SESSION_SECRET}
      providers:
        oidc:
          production:
            clientId: ${KEYCLOAK_CLIENT_ID}
            clientSecret: ${KEYCLOAK_CLIENT_SECRET}
            metadataUrl: ${KEYCLOAK_BASE_URL}/realms/${KEYCLOAK_RHDH_REALM}
            prompt: auto
    catalog:
      providers:
        keycloakOrg:
          default:
            baseUrl: ${KEYCLOAK_BASE_URL}
            clientId: ${KEYCLOAK_CLIENT_ID}
            clientSecret: ${KEYCLOAK_CLIENT_SECRET}
            realm: ${KEYCLOAK_RHDH_REALM}
            loginRealm: ${KEYCLOAK_RHDH_REALM}
            schedule:
              frequency: { minutes: 5 }
    integrations:
      gitlab:
        - host: gitlab.{openshift_cluster_ingress_domain}
          token: ${GITLAB_TOKEN}
          apiBaseUrl: https://gitlab.{openshift_cluster_ingress_domain}/api/v4
          baseUrl: https://gitlab.{openshift_cluster_ingress_domain}
```

Sample output

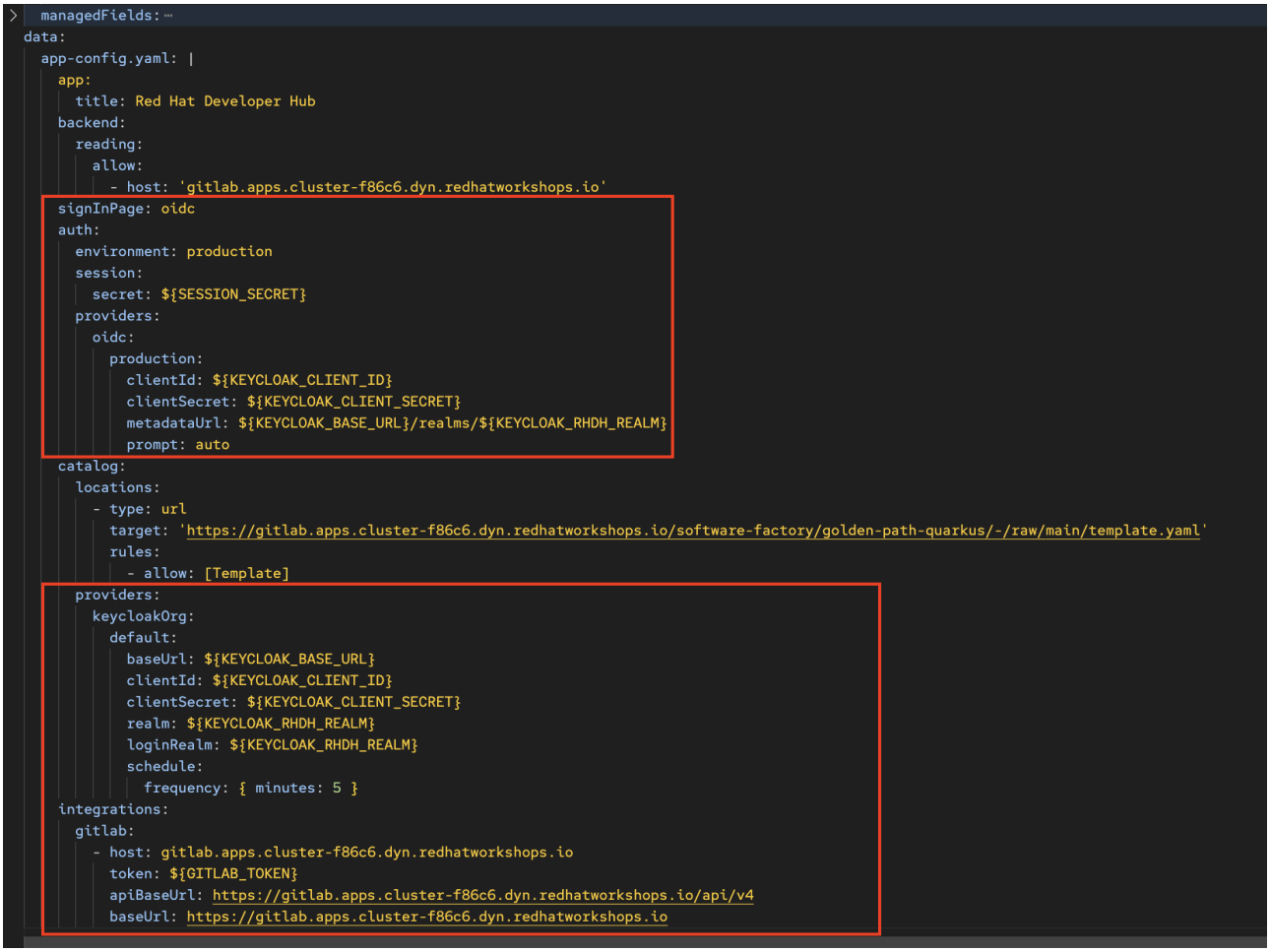


Figure 31. Sample rhdh-config changes(Click the image to view it in full size in a new browser tab)

Click **Save**.

7. Wait until the Developer Hub pod returns to the **Running** state.

```
oc get pods -n setup-rhdh -w
```



The schedule frequency would be set to something higher in production (12 hours) and is set low for demo purposes. Other options like userQuerySize or groupQuerySize are also good to set.

1. Click **[Save]**. Wait until the Developer Hub pod returns to the **Running** state.

```
oc get pods -n setup-rhdh -w
```

Create Dynamic Plugins ConfigMap

Create a dynamic plugins ConfigMap to enable the Keycloak Org sync plugin and the gitlab scaffolder plugin for our template

```
oc create -f - <<EOF
kind: ConfigMap
```

```
apiVersion: v1
metadata:
  name: dynamic-plugins-rhdh
  namespace: setup-rhdh
data:
  dynamic-plugins.yaml: |
    includes:
      - dynamic-plugins.default.yaml
    plugins:
      - package: './dynamic-plugins/dist/backstage-community-plugin-catalog-backend-
module-keycloak-dynamic'
        disabled: false
      - package: './dynamic-plugins/dist/backstage-plugin-scaffolder-backend-module-
gitlab-dynamic'
        disabled: false
EOF
```

Update the Backstage CR

Add the **dynamic-plugins-rhdh** config map so we can enable the keycloak Org sync

1. Open Openshift console and switch to the **setup-rhdh** project.

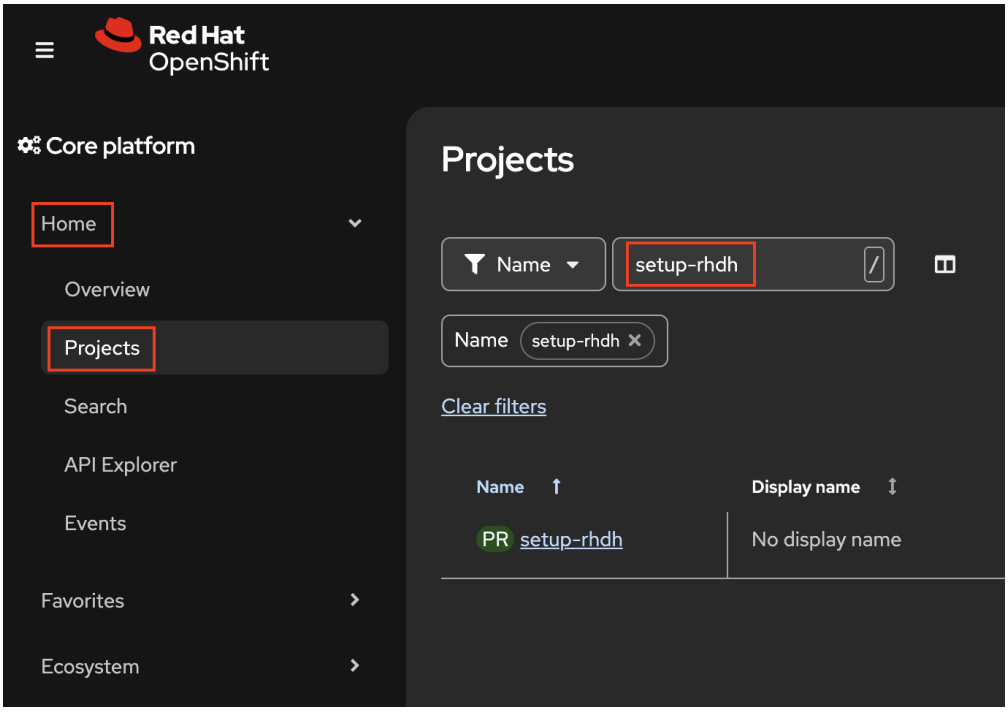


Figure 32. Openshift Console(Click the image to view it in full size in a new browser tab)

2. Go to **[Ecosystem]** → **[Installed Operators]**. Select **Red Hat Developer Hub** Operator.

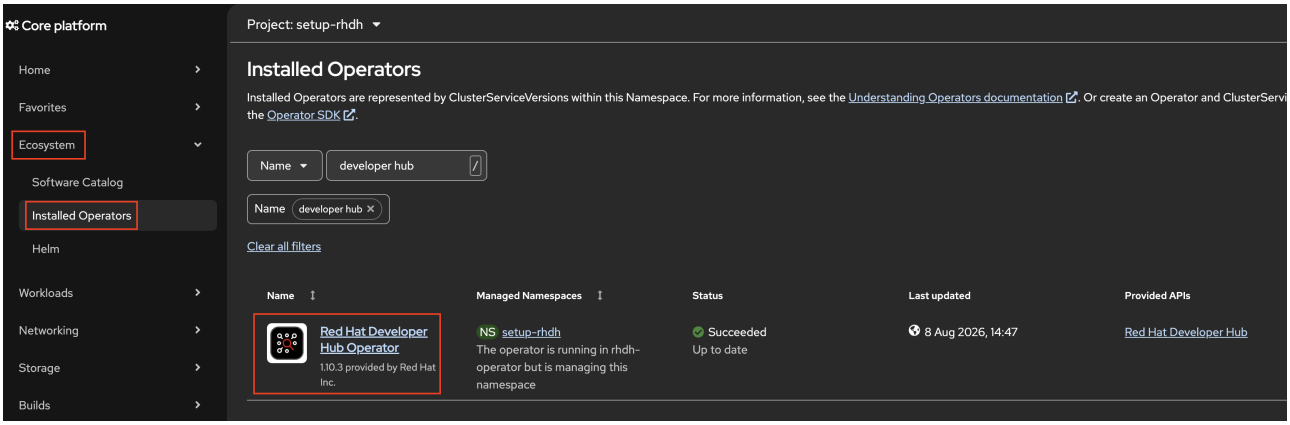


Figure 33. Red Hat Developer Hub Operator(Click the image to view it in full size in a new browser tab)

3. Click tab **[Red Hat Developer Hub]** → click **[rhdh]** instance created.

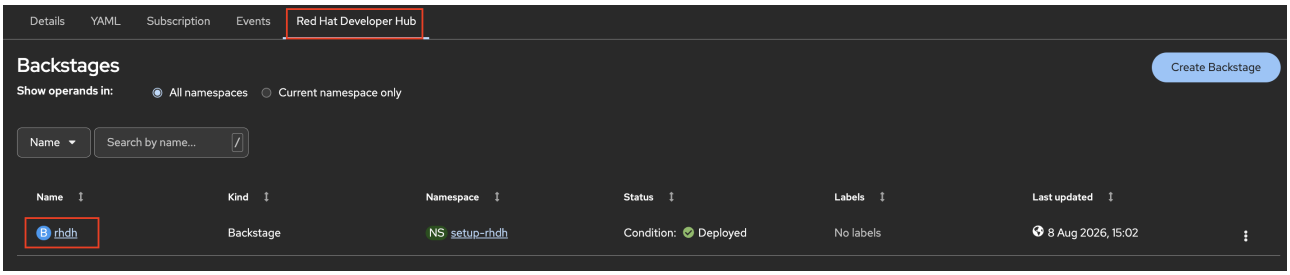


Figure 34. rhdh instance(Click the image to view it in full size in a new browser tab)

4. Click **[Actions]** → **[Edit Backstage]**

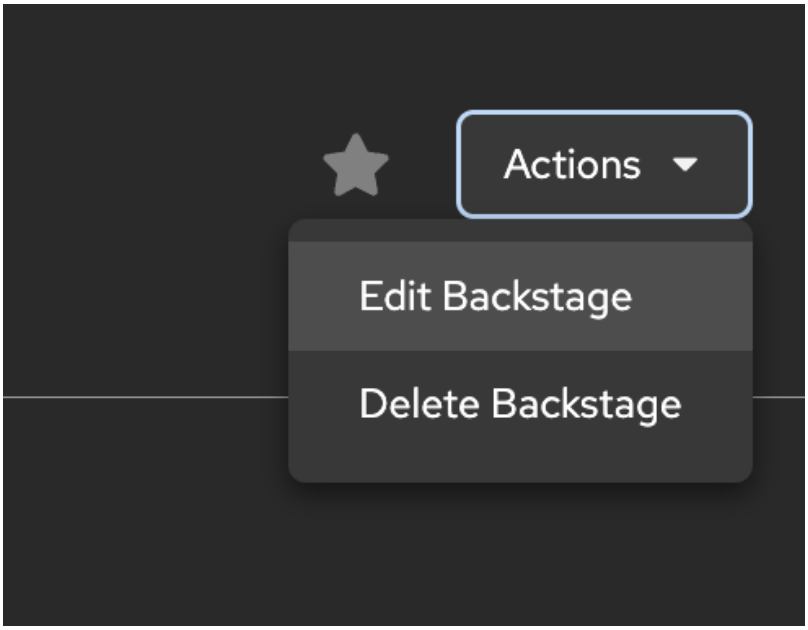


Figure 35. rhdh instance(Click the image to view it in full size in a new browser tab)

5. Switch to **YAML** view, under **spec.application**, add the **extraEnvs** & **dynamicPluginsConfigMapName** blocks below. Keep the existing appConfig and route sections from the initial deployment.

```
    extraEnvs:
      secrets:
        - name: rhdh-auth-secret
```

```
    dynamicPluginsConfigMapName: dynamic-plugins-rhdh
```

Sample Output

```
spec:
  application:
    extraEnvs:
      secrets:
        - name: rhdh-auth-secret
    dynamicPluginsConfigMapName: dynamic-plugins-rhdh
    appConfig:
      configMaps:
        - name: rhdh-config
      mountPath: /opt/app-root/src
```

Figure 36. Environment Vairable Settings(Click the image to view it in full size in a new browser tab)

Verify Keycloak sign-in

Wait until the Developer Hub pod returns to the **Running** state.

```
oc get pods -n setup-rhdh -w
```

Retrieve the Developer Hub route and open the route URL in your browser:

```
oc get route backstage-rhdh \
  -n setup-rhdh \
  -o jsonpath='https://{.spec.host}{"\n"}'
```

On the login page, choose OIDC.

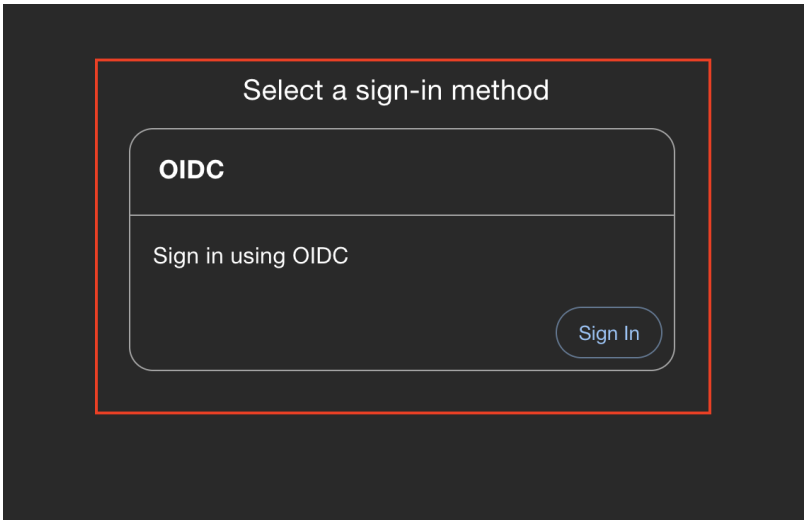


Figure 37. OIDC login page(Click the image to view it in full size in a new browser tab)

Sign in with your Keycloak credentials (for example platform-developer / openshift).

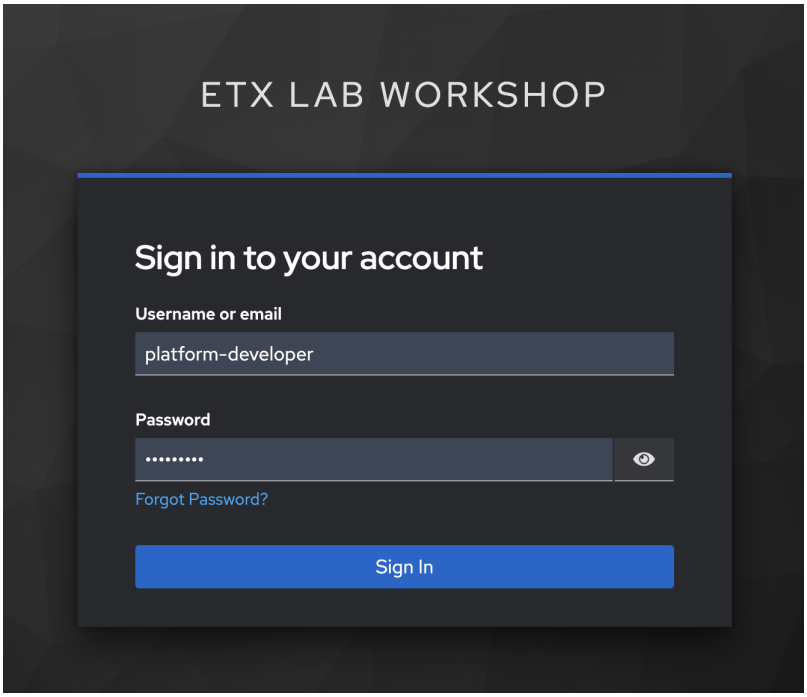


Figure 38. ETX Lab Login Page(Click the image to view it in full size in a new browser tab)

You should land on the Developer Hub home page authenticated as your Keycloak user, not as Guest.

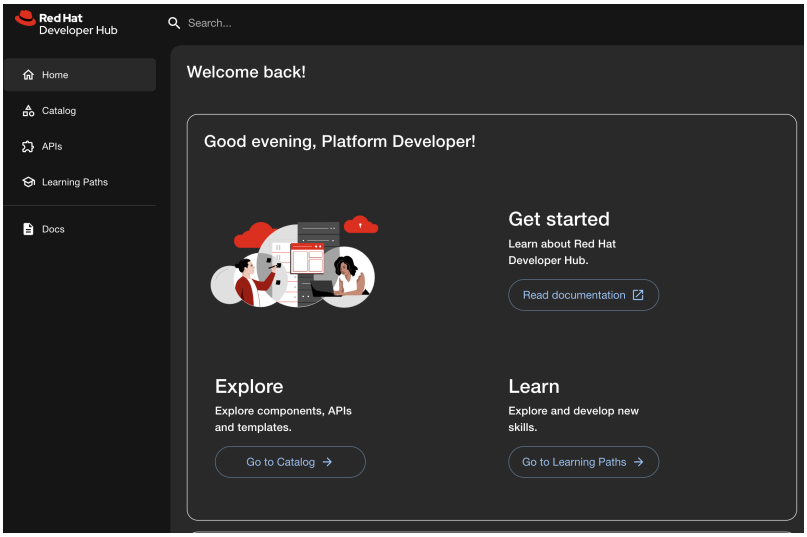


Figure 39. Developer Hub Home Page(Click the image to view it in full size in a new browser tab)



The signed-in user must exist in the Software Catalog (or be mapped via your org setup, which is what we configured with the keycloakOrg provider) to use some catalog features. For template creation in this lab, guest access is no longer available after this change.

Test the Golden Path Template

Verify that authenticated users can create new applications.

1. Open **[Catalog]** → **[Kind]** → **[Templates]** and select **[Quarkus Application (Golden Path)]**

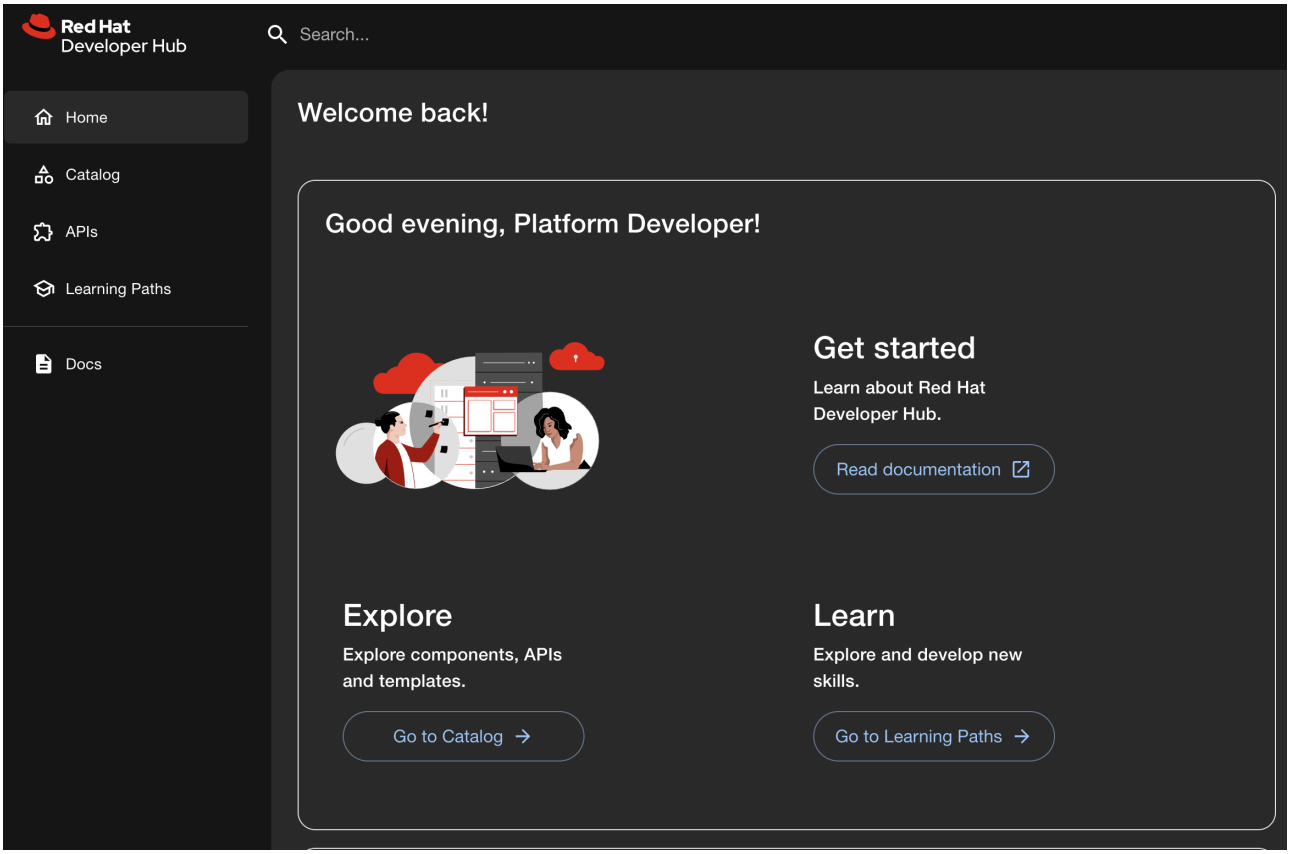


Figure 40. Developer Hub Home Page(Click the image to view it in full size in a new browser tab)

2. Click **[Launch Template]**

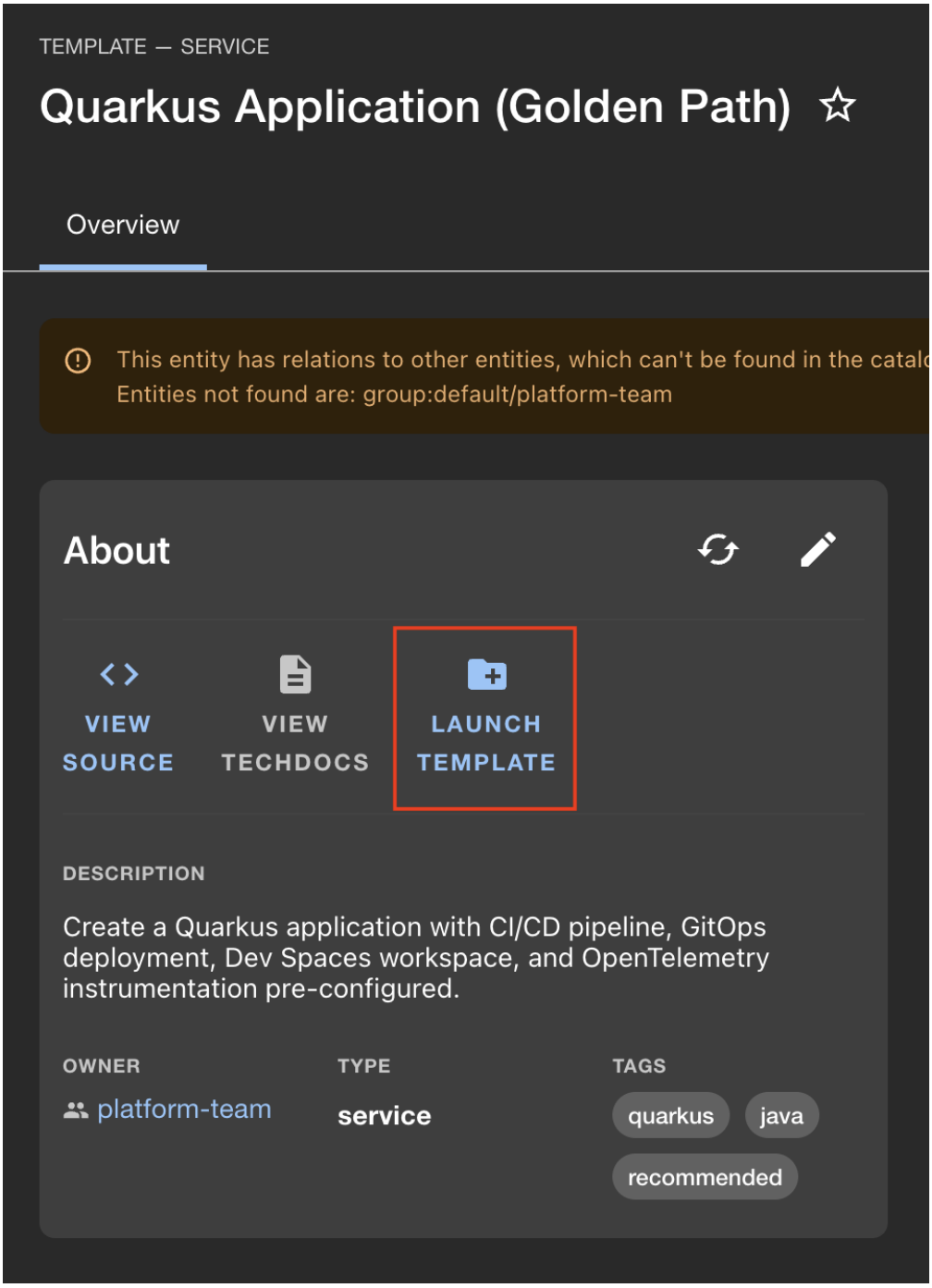


Figure 41. Launch Template(Click the image to view it in full size in a new browser tab)

3. Enter the required project information.

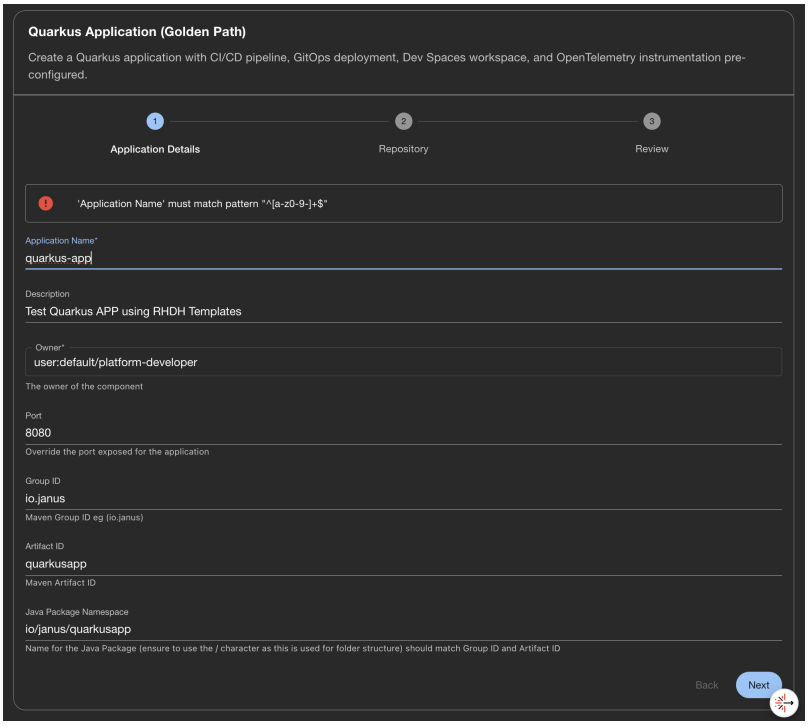


Figure 42. Sample Project Information Page 1(Click the image to view it in full size in a new browser tab)

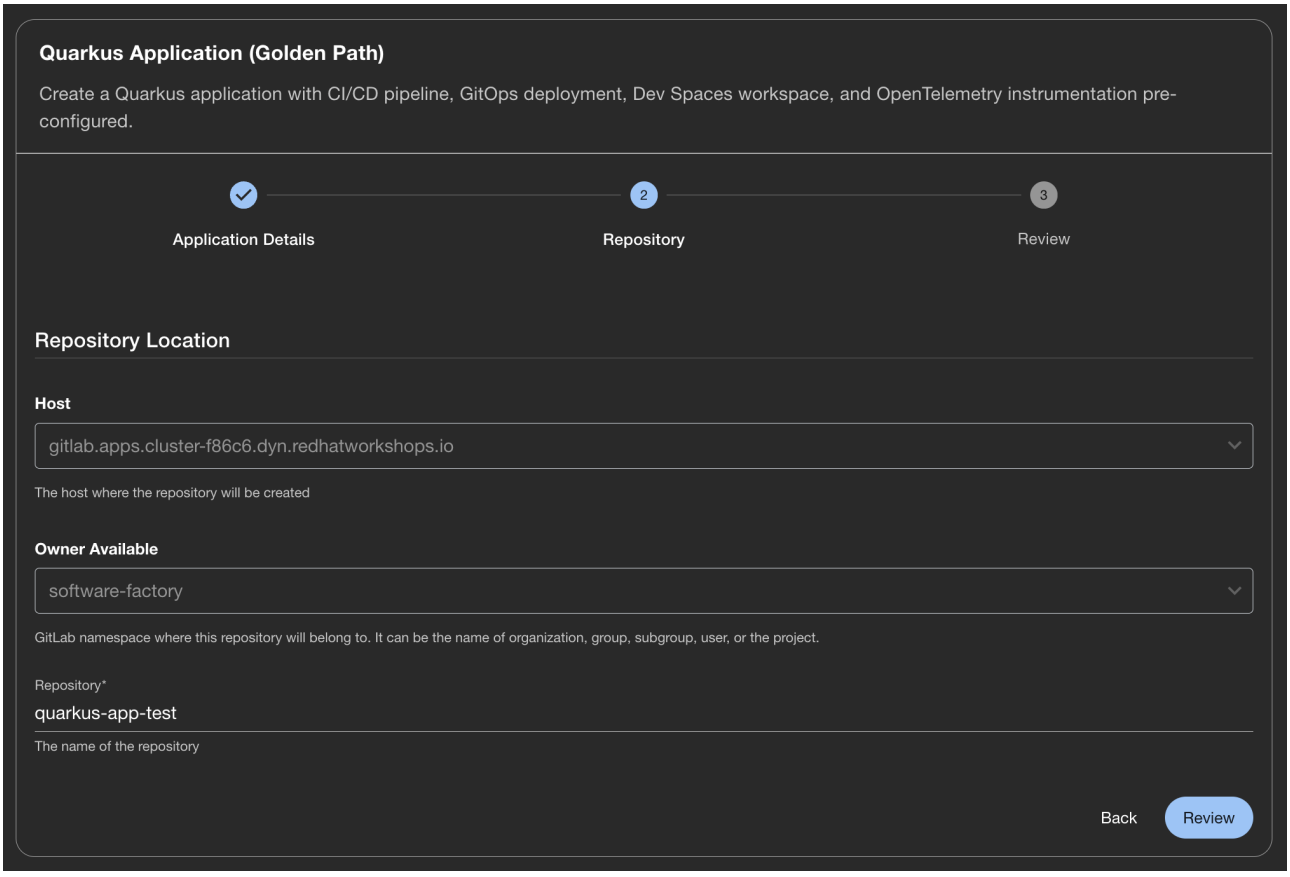


Figure 43. Sample Project Information Page 2(Click the image to view it in full size in a new browser tab)

4. Review all the information and click **[Create]**

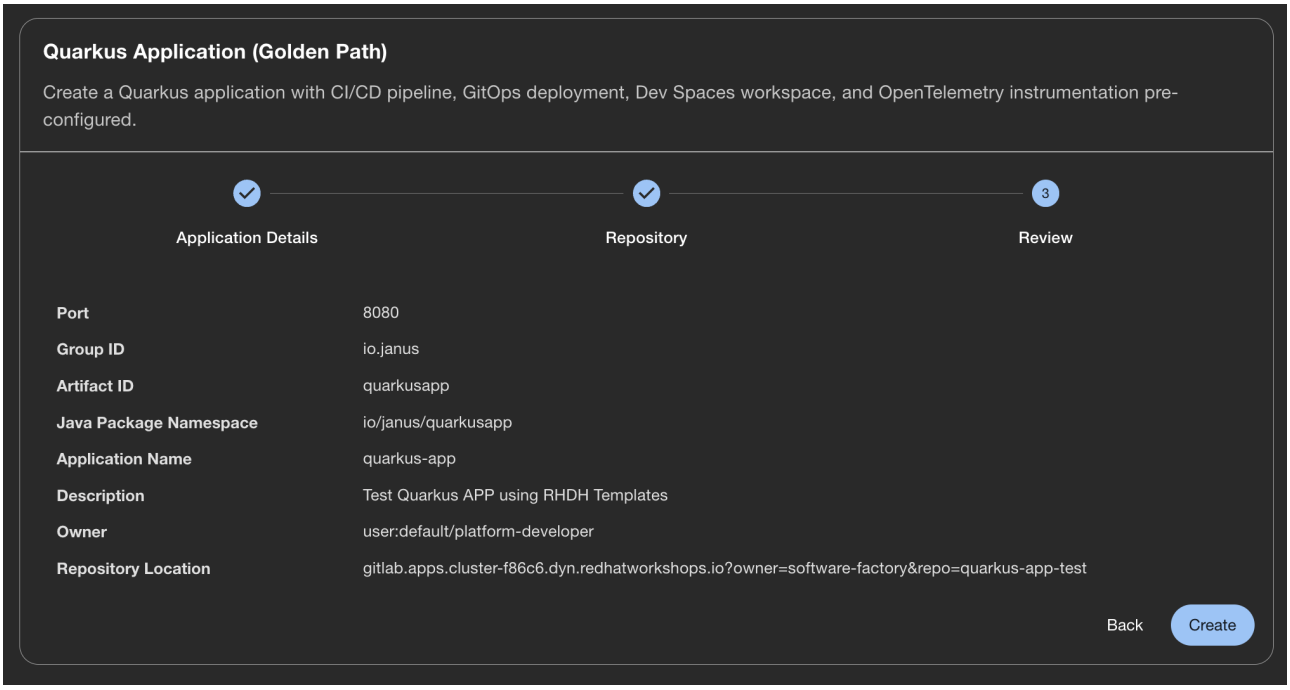


Figure 44. Sample Project Information Page 3(Click the image to view it in full size in a new browser tab)

5. Once the template run finishes, the output should be something like below:

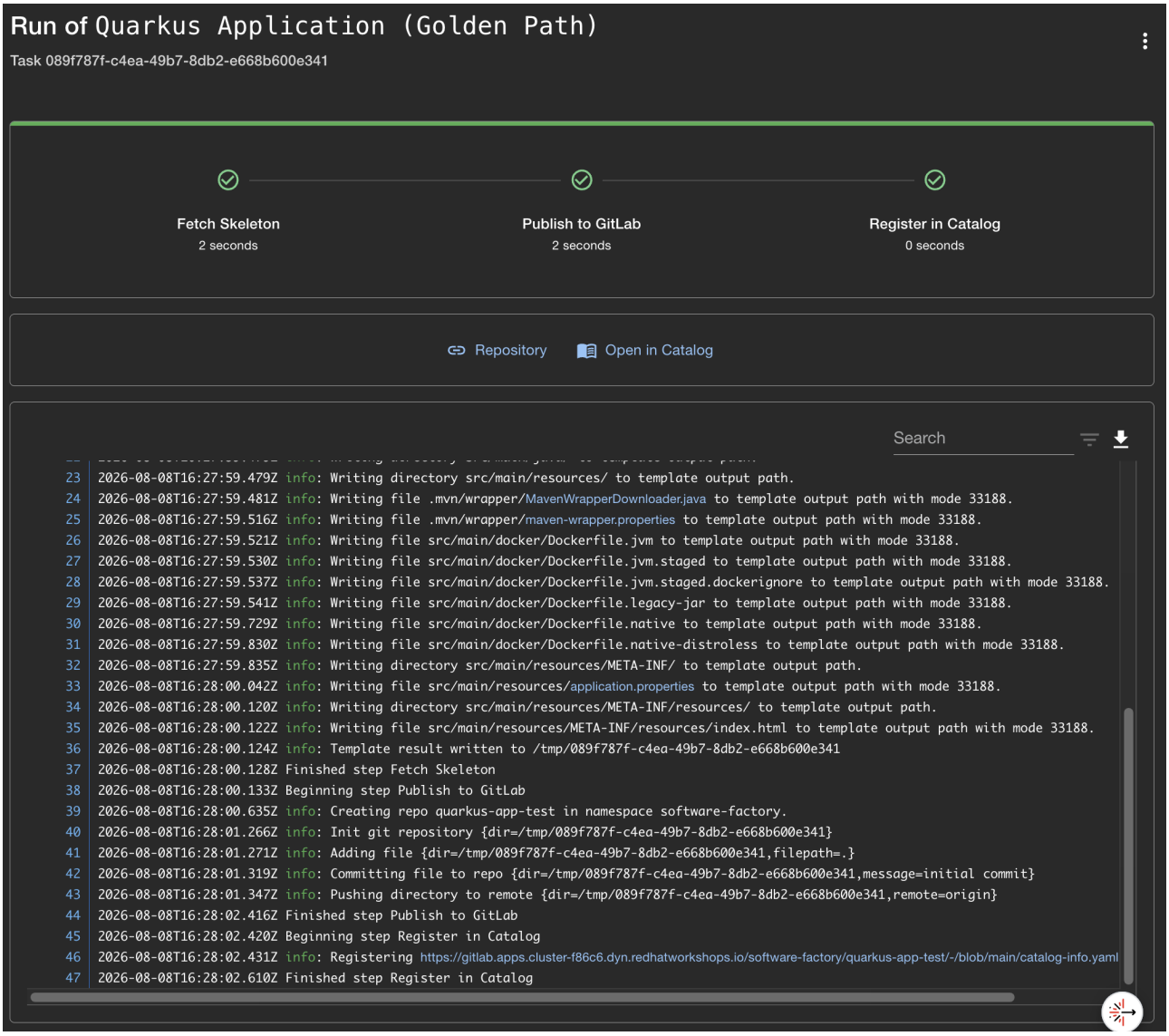


Figure 45. Sample Template Run Output(Click the image to view it in full size in a new browser tab)

Verify that:

- A new GitLab repository is created and the application is registered in the Software Catalog.

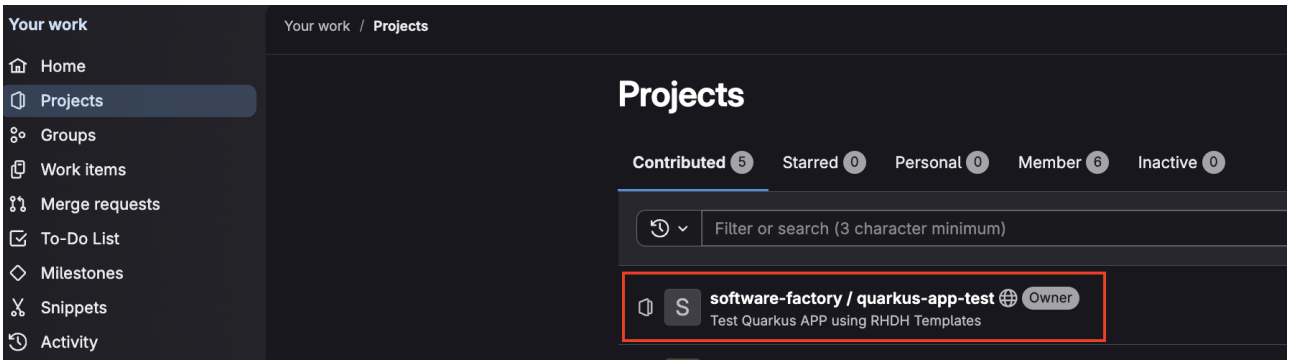


Figure 46. Newly created git repository(Click the image to view it in full size in a new browser tab)

- The generated repository contains:
  - Devfile
  - Tekton pipeline
  - GitOps manifests

- The catalog component provides a link to launch a Dev Spaces workspace.

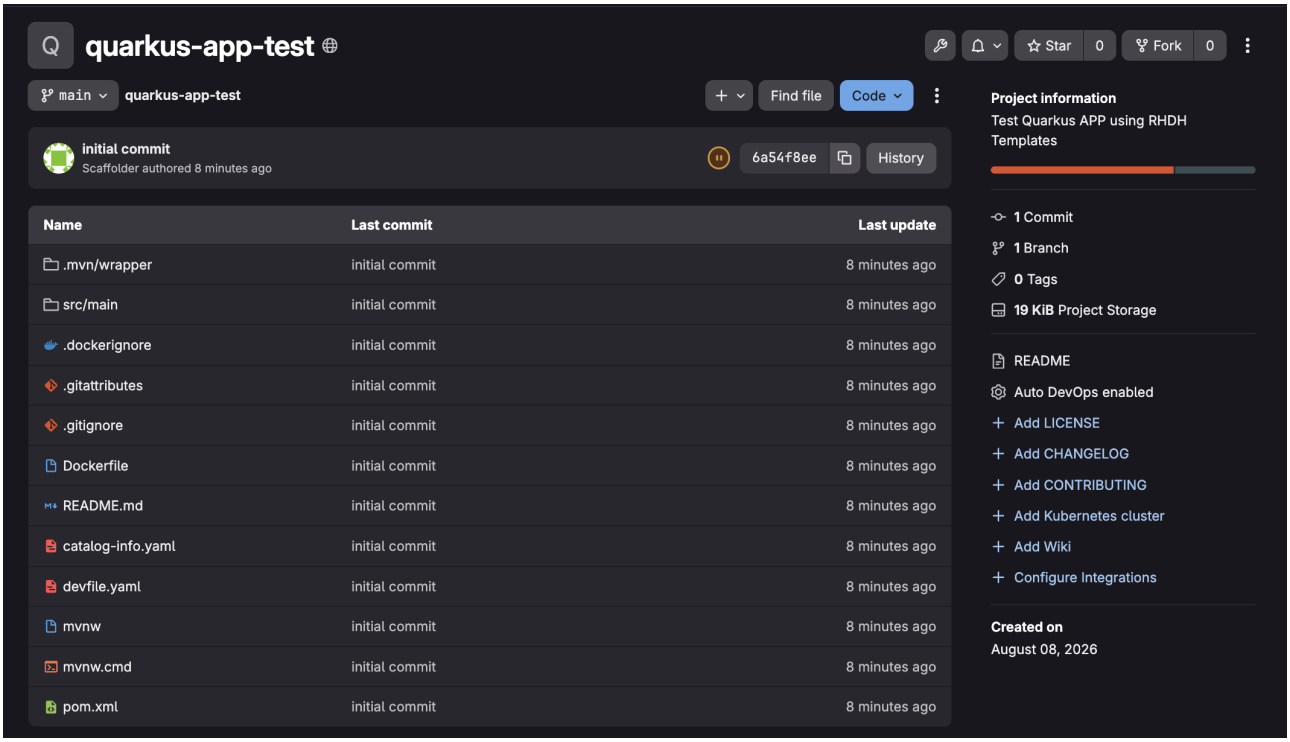


Figure 47. Git repository content(Click the image to view it in full size in a new browser tab)

```
<div class="key-card">
```

Key Takeaway

Red Hat Developer Hub now uses OpenShift OAuth for authentication, allowing developers to securely access Software Templates and the Software Catalog using their enterprise identity while maintaining a production-ready authentication model.

```
</div>
```

Lab Summary

In this module, you installed and configured **Red Hat Developer Hub on OpenShift** and established it as the Internal Developer Portal for the Software Factory. You created a reusable **Software Template (Golden Path)** that automates application project setup, registered the template in the Developer Hub Software Catalog, and verified its metadata and input form.

You then replaced temporary Guest authentication with **OpenShift OAuth**, enabling developers to securely access Developer Hub using their OpenShift identities. Finally, you validated the complete self-service experience by using an authenticated user to access and test the Golden Path template.

Knowledge Check

<iframe type="text/javascript" src="_attachments/questions.html" style="width: 100%; max-width: 1024px; height: 800px" allowfullscreen webkitallowfullscreen mozAllowFullScreen allow="autoplay *; fullscreen *; encrypted-media *" frameborder="0"></iframe>

Summary and What's next?

Congratulations! You have successfully established the foundation of your DevEx with Red Hat Developer Hub.

In this module, you learned how Red Hat Developer Hub (RHDH) provides a unified Internal Developer Portal for the Software Factory. You explored Developer Experience, the role of Developer Hub, Software Templates (Golden Paths), and the Software Catalog in reducing developer cognitive load and enabling self-service application delivery.

You also created and registered a reusable Software Template, integrated it with the Software Catalog, and configured OpenShift OAuth for secure authentication. By the end of the module, you established a foundation for standardized, self-service application creation through Developer Hub.

In the next module, you'll explore how Migration Toolkit for Applications (MTA) and Red Hat Developer Lightspeed can accelerate enterprise application modernization through automated analysis and AI-assisted migration recommendations.